

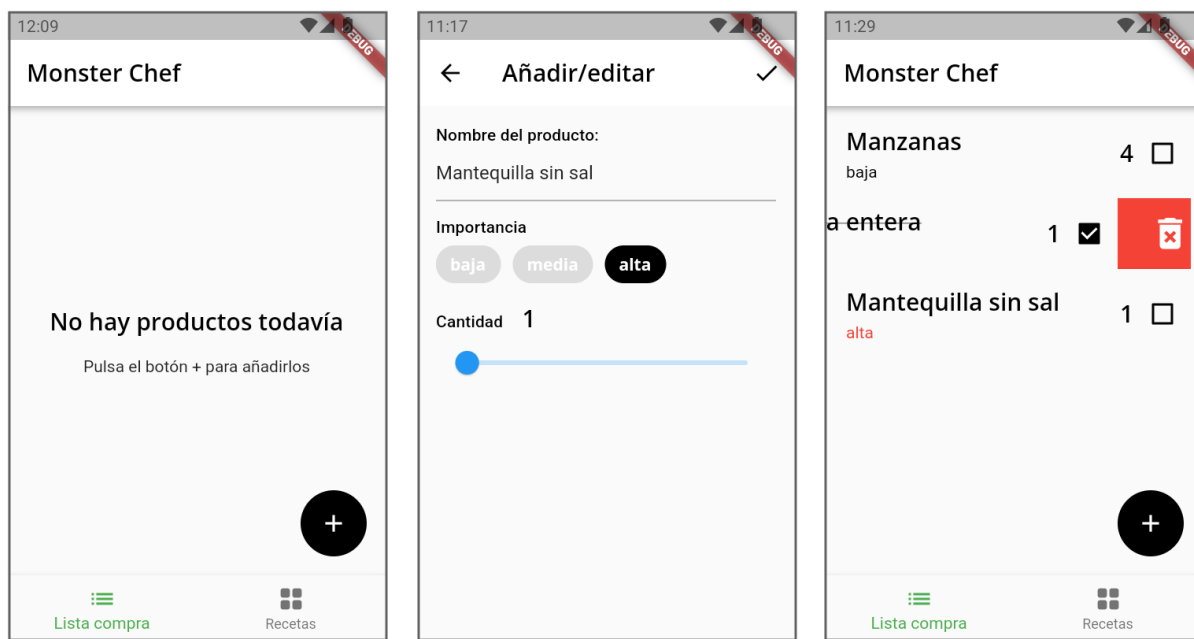
Flutter 2: Lista de la compra

Contenidos

- Flutter 2: Lista de la compra
 - ¿Qué voy a hacer en esta sesión?
 - Pasos previos
 - 1 – Escribir la clase aplicación y conectarla con la función main
 - 2 – Escribir la pantalla principal de la aplicación
 - 3 – Añadir la barra de navegación
 - 4 – Conectar botones y pantallas
 - Escribir las clases para la pantalla de cada categoría
 - Establecer la conexión
 - 5 – Temas claro y oscuro
 - Escribir un tema personalizado para la aplicación
 - Añadir soporte para la biblioteca Google Fonts
 - Usar el tema en la aplicación
 - 6 – Diseño de las pantallas
 - 7 – Pantalla para cuando la lista de la compra está vacía
 - 8 – Mostrar la pantalla para cuando la lista de la compra está vacía
 - 9 – El patrón estado en Flutter
 - 10 – Estado mutable para la lista de la compra
 - Representar un producto
 - Representar la lista de la compra
 - 11 – Proveedor de acceso a la lista de la compra
 - 12 – Consumir los cambios
 - 13 – Pantalla para añadir productos a la lista de la compra
 - 14 – Mostrar la pantalla para añadir/editar productos
 - 15 – Completar la pantalla para añadir productos
 - Añadir el campo texto para el nombre
 - Añadir vistas para seleccionar la importancia del producto
 - Añadir el selector para la cantidad
 - 16 – Añadir los productos a la lista de la compra
 - 17 – Pantalla para mostrar la lista de la compra con productos
 - 18 – Detallar la información de los productos
 - 19 – Marcar una tarea como completada
 - 20 – Editar un producto ya añadido
 - 21 – Eliminar un producto
 - Mini ejercicios
 - Apéndice A: Listado completo del tema personalizado

¿Qué voy a hacer en esta sesión?

En esta sesión escribiremos una aplicación para gestionar una lista de la compra. Es parte de una aplicación más grande llamada Monster Chef, en la que también se podrán consultar recetas de cocina. Esta parte de las recetas de cocina es el tema de la siguiente sesión de prácticas. El aspecto de la aplicación terminada es este:



Esta aplicación es más compleja que el conversor de unidades. Mientras la escribimos paso a paso aprenderemos:

- A usar el patrón de diseño de software llamado **observador** y la forma en que se implementa en Flutter: **Provider – Consumer**
- A usar el patrón de interacción llamado **Información anidada** que en Flutter se programa mediante las clases `Navigator` y `MaterialPageRoute`
- A usar el patrón de interacción llamado **Barra de navegación** que en Flutter se programa mediante una vista `BottomNavigationBar` gestionada por la vista `Scaffold` que permite organizar los elementos principales de la interfaz de usuario de una aplicación móvil.
- A definir un tema personalizado, que pueda reutilizarse para usar los mismos esquemas de colores y tipos de letra en distintas aplicaciones. También aprenderemos a utilizar los modos claro y oscuro del sistema operativo en la aplicación de forma que se adapte al modo seleccionado por el usuario en su dispositivo móvil.
- A gestionar el contenido de una vista `TextField` de forma más flexible que la aprendida en la sesión anterior, mediante un `TextEditingController`
- A añadir dependencias a un proyecto Flutter editando el archivo de configuración: `pubspec.yaml` y usando la orden de Flutter: `flutter pub get`
- A utilizar el paquete `Uuid` para asignar identificadores únicos para los distintos objetos de la aplicación.
- A utilizar entre otros los widgets de Flutter: `FloatingActionButton`, `ListView`, `Row`, `Container`, `Wrap`, `ChoiceChip`, `Slider`, `InkWell`, `Dismissible`, `SnackBar`

Pasos previos

Comenzaremos creando una nueva aplicación Flutter llamada `flutter_sesion_2_3` dentro de la carpeta `flutter_projects`

Nota: El nombre `flutter_sesion_2_3` hace referencia a que en esta sesión comenzaremos a escribir una aplicación que terminaremos en la siguiente sesión.

Como sabemos, Flutter rellena el archivo `main.dart` con algunas clases. Eso no es una buena idea, cada clase debería estar en su propio archivo. Por eso, lo primero que haremos será borrar el contenido del archivo `main.dart` y dejar solo una función `main` vacía.

Nota: Si no te acuerdas de como se hace, puedes consultar el guión de la sesión 4 (Flutter I). El proceso que necesitamos aquí es el mismo que el que se describe al comienzo de esa sesión.

1 – Escribir la clase aplicación y conectarla con la función main

Toda aplicación Flutter tiene una clase aplicación que es subclase de `StatelessWidget` y que, cuando se construye, devuelve una vista de tipo `MaterialApp`. Por tanto, añadiremos al proyecto un archivo llamado `monster_chef_app.dart` y en él declararemos la clase aplicación `MonsterChefApp` que devolverá en el método `build`:

```
return MaterialApp(  
  title: 'Monster Chef Application',  
  theme: ThemeData(primarySwatch: Colors.blue,),  
  home: const Placeholder(),  
);
```

Después la conectaremos con la función `main` tal y como hicimos en la sesión anterior.

2 – Escribir la pantalla principal de la aplicación

En el paso anterior, pasamos a la propiedad `home:` del widget `MaterialApp` un objeto de la clase `Placeholder` para que la aplicación pudiese compilar. Ahora es el momento de escribir una vista completa para que sea la página principal de la aplicación.

Añadiremos un nuevo archivo al proyecto al que llamaremos `monster_chef_pagina_principal.dart` y en él declararemos una clase `MonsterChefPaginaPrincipal` que sea subclase de `StatefulWidget`

En este caso, se ha elegido una subclase de `StatefulWidget` porque la página principal va a tener varias categorías de navegación y para poder cambiar de una a otra se necesita poder gestionar el estado mutable.

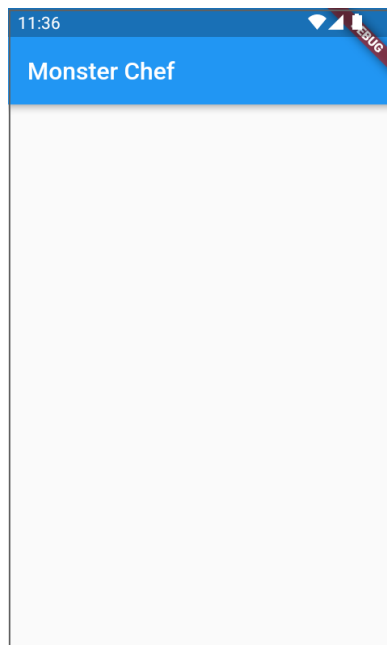
El método `build` del estado asociado a la página principal debe devolver una vista `Scaffold` básica:

```
return Scaffold(  
  appBar: AppBar(title: Text(widget.titulo)),  
  body: Center(),  
);
```

Nota: Fíjate que en esta ocasión el título de la barra de aplicación lo hemos declarado como una propiedad de la clase `MonsterChefPaginaPrincipal`. Esa propiedad se pasa en el constructor de esa clase cuando se construye, y desde el estado asociado se accede a la propiedad mediante la propiedad `widget`.

Si no se te ocurre cómo hacerlo, siempre puedes escribir la cadena `'Monster Chef'` como argumento de la vista `Text`.

Si ejecutamos la aplicación en un emulador, obtendremos una pantalla como esta:



Aspecto de la aplicación tras el paso 2. Podemos ver la barra de aplicación en la parte superior con el título.

3 – Añadir la barra de navegación

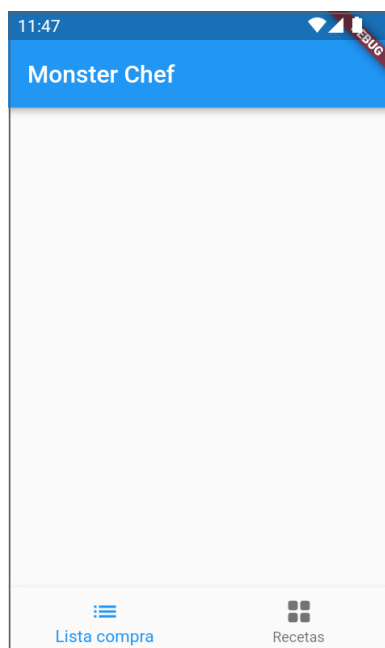
En esta aplicación usaremos dos categorías, una donde consultar recetas y otra donde gestionar una lista de la compra. Por eso usaremos el patrón de interacción llamado **barra de navegación** para gestionar la navegación por esas dos categorías.

En Flutter, el patrón barra de navegación se programa añadiendo la propiedad `bottomNavigationBar` a la vista `Scaffold`. A esta propiedad hay que pasarle una vista de clase `BottomNavigationBar`.

Justo debajo de la propiedad `body:` de la clase `Scaffold` escribiremos:

```
bottomNavigationBar: BottomNavigationBar(  
  items: const <BottomNavigationBarItem>[  
    BottomNavigationBarItem(  
      icon: Icon(Icons.list),  
      label: 'Lista compra',  
    ),  
    BottomNavigationBarItem(  
      icon: Icon(Icons.grid_view_rounded),  
      label: 'Recetas',  
    ),  
  ],  
)
```

Si ejecutamos la aplicación ahora podremos ver una barra de navegación en la parte inferior, con los dos iconos correspondientes a las dos categorías de la aplicación.



Aspecto de la aplicación tras el paso 3.

Ahora podemos ver la barra de navegación con las dos categorías en la parte inferior.

La aplicación no está completa todavía, pero antes de seguir veamos algunos aspectos interesantes.

1. La vista `Scaffold` es la que organiza en Flutter los elementos principales de la interfaz de usuario. En este caso, la barra de navegación. Para añadir una barra de navegación a la aplicación solo hay que usar la propiedad `bottomNavigationBar:` de la vista `Scaffold`
2. Una de las propiedades de la barra de navegación es `items:` que como su nombre indica espera una lista de vistas de clase `BottomNavigationBarItem`. Hay que añadir uno por cada botón que se quiera colocar en la barra de navegación (uno por cada categoría principal de la aplicación)

3. Dentro de cada `BottomNavigationBarItem` hay dos propiedades: `icon` donde se define el icono que se muestra en el botón, y `label`: donde se muestra la etiqueta asociada al botón.

Importante: Las guías de estilo, tanto de Android, como de iOS, recomiendan que se usen iconos y etiquetas de texto, para los botones de las barras de navegación, por tanto **siempre deberías usar las dos propiedades** en cada `BottomNavigationBarItem`

4. La lista de `BottomNavigationBarItem` se hace constante ya que los botones de esa barra siempre se van a ver igual, no cambian, por tanto si se hacen constantes, Dart los va a gestionar como **instancias canónicas** y no se van a reconstruir en memoria cada vez que se refresque la interfaz.

Nota: Cada vez que un usuario toque un botón de la barra de navegación se tiene que refrescar la interfaz. Son muchas operaciones de refresco. Todos los recursos que se puedan ahorrar en creación/destrucción de objetos en memoria ayudan a hacer mas eficiente la aplicación.

4 – Conectar botones y pantallas

La aplicación tiene una barra de navegación, pero todavía no funciona. Si se pulsa en la categoría `Recetas` no ocurre nada. Es normal ya que aún no hemos construido las pantallas asociadas a la lista de la compra y a las recetas y no las hemos conectado con la barra de navegación.

Escribir las clases para la pantalla de cada categoría

Añadiremos dos nuevos archivos al proyecto llamados: `lista_compra_pantalla.dart` y `recetas_pantalla.dart` y en ellos escribiremos dos nuevas clases `ListaCompraPantalla` y `RecetasPantalla` ambas subclases de `StatelessWidget`

Contenido del archivo `lista_compra_pantalla.dart`:

```
import 'package:flutter/material.dart';

class ListaCompraPantalla extends StatelessWidget {
  const ListaCompraPantalla({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text('Lista de la compra 🛒'),
    );
  }
}
```

Contenido del archivo: `recetas_pantalla.dart`

```
import 'package:flutter/material.dart';

class RecetasPantalla extends StatelessWidget {
  const RecetasPantalla({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text('Galería de recetas 🍴'),
    );
  }
}
```

Establecer la conexión

Queremos que cada vez que el usuario pulse en uno de los botones de la barra de navegación, se muestre la pantalla asociada. Para ello, en la clase: `_MonsterChefPaginaPrincipalState` haremos lo siguiente:

1. Añadir el estado mutable:

```
int _categoriaActiva = 0;
```

Como su nombre indica, contendrá el índice de la categoría de navegación que se haya seleccionado en ese momento. Como valor inicial le daremos el valor `0`. Forma parte del estado mutable que se quiere gestionar en la clase `MonsterChefPaginaPrincipal` por tanto, cualquier cambio que se le haga deberá estar dentro de una llamada al método `setState` que hereda la clase `_MonsterChefPaginaPrincipalState`

2. Añadir una propiedad de clase (`static`) que contendrá la lista de páginas asociadas a cada categoría y que se han escrito en el paso anterior:

```
static var paginas = <Widget>[
  ListaCompraPantalla(),
  RecetasPantalla(),
];
```

Esta lista contiene las pantallas a las que dan acceso los botones de la barra de navegación ordenados de la misma forma. En la posición `0` de la lista está la pantalla asociada al primer botón, etc.

Nota: La lista `paginas` se podría haber declarado como una propiedad de instancia en lugar de como una propiedad de clase (`static`) ya que en todo momento solo va a haber en memoria una instancia de la página principal. ¿Adivinas por qué se ha optado por hacerla de clase?

3. Buscar la propiedad `body:` de la vista `Scaffold` que actualmente tiene este aspecto:

```
body: Center(),
```

y sustituirla por esta otra:

```
body: paginas[_categoriaActiva],
```

Cada vez que se redibuje la vista `MonsterChefPaginaPrincipal`, su estado asociado `_MonsterChefPaginaPrincipalState` dibujará como cuerpo en la vista `Scaffold`, bien una pantalla de tipo `ListaCompraPantalla`, bien de tipo `RecetasPantalla`, en función del valor de la propiedad `_categoriaActiva`

4. Añadir un método privado para gestionar el cambio de pantalla:

```
void _alPulsar(int indice) {  
  setState(() => _categoriaActiva = indice);  
}
```

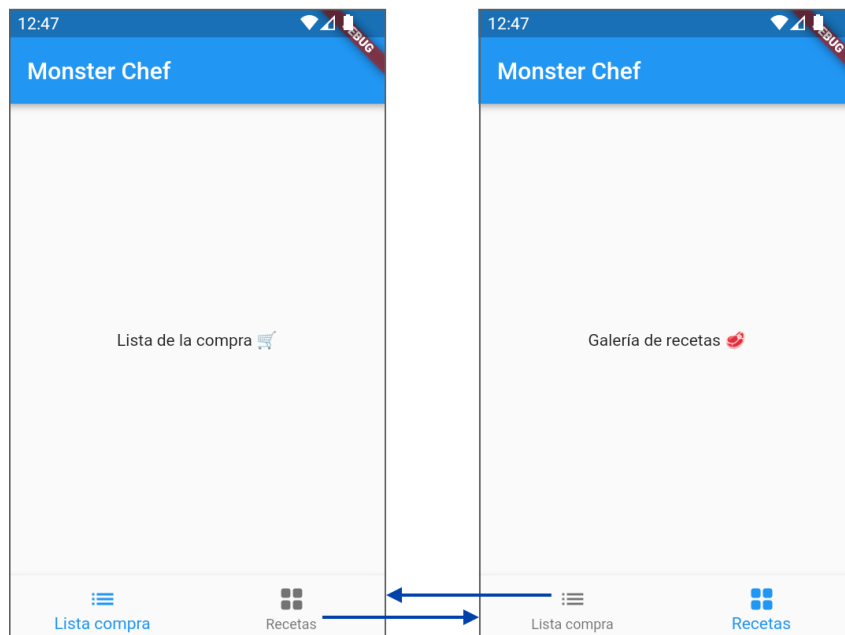
Este método, cambia el valor de la propiedad `_categoriaActiva` y le asigna el valor `indice`, pero lo hace dentro de una llamada a `setState` con lo que informa a Flutter que parte del estado mutable de la aplicación ha cambiado y por tanto debe refrescar las vistas a partir de la vista actual que es `MonsterChefPaginaPrincipal`

5. Lo último que hay que hacer en la clase `_MonsterChefPaginaPrincipalState` es llamar al método declarado en el paso anterior, cada vez que el usuario toque en el icono de alguna categoría. Para ello buscaremos el constructor del `BottomNavigationBar` y sobre la propiedad `items:` añadiremos dos nuevas propiedades:

```
currentIndex: _categoriaActiva,  
onTap: _alPulsar,
```

Con esto ya está terminada la barra de navegación. Es completamente funcional, aunque las pantallas a las que nos lleva, todavía no están completas.

Nota: En esta sesión de prácticas se completará la pantalla para gestionar la lista de la compra. La pantalla para ver la galería de recetas se completará en la siguiente sesión de prácticas.



Aspecto de la aplicación tras el paso 4. Ahora la barra de navegación es completamente funcional aunque las pantallas a las que nos lleva no están terminadas.

5 – Temas claro y oscuro

A partir de la versión 12 de iOS y de la versión 10 de Android, se puede escoger entre un tema claro y uno oscuro a nivel del Sistema Operativo. El tema claro es más adecuado cuando se usa el móvil de día con más luz natural y el tema oscuro lo es cuando se usa de noche con poca iluminación.

Hacer que los colores de la interfaz de nuestra aplicación Flutter se adapte a esos cambios de tema en el Sistema Operativo es muy sencillo.

Vamos a hacer una pequeña pausa en el desarrollo de nuestra lista de la compra para añadir esta característica a nuestra aplicación Flutter.

Escribir un tema personalizado para la aplicación

Importante: Escribir un tema personalizado para una aplicación es tedioso, porque hay que definir los colores y estilos de letra para muchos elementos, pero una vez terminado, se puede reutilizar en varias aplicaciones y simplifica muchísimo el trabajo.

Añadiremos una nueva clase al proyecto y la llamaremos `MonsterChefTema`. Esa clase no extiende a ningún widget de Flutter ya que la usaremos para encapsular unas propiedades de clase y unos métodos de clase que devuelven los temas claro y oscuro de la aplicación.

Nota: El código completo de la clase `MonsterChefTema` se encuentra al final de este guión de prácticas, en el apéndice A.

Añadiremos a la clase `MonsterChefTema` una primera propiedad de clase llamada `temaTextoClaro` que como su nombre indica contendrá los ajustes de texto para el tema claro.

```
static TextTheme temaTextoClaro = TextTheme(  
  bodyLarge: GoogleFonts.openSans(  
    fontSize: 14.0,  
    fontWeight: FontWeight.w700,  
    color: Colors.black,  
  ),  
  displayLarge: GoogleFonts.openSans(  
    fontSize: 32.0,  
    fontWeight: FontWeight.bold,  
    color: Colors.black,  
  ),  
  // - Aquí los demás estilos de texto  
);
```

y otra propiedad de clase llamada `temaTextoOscuro` de forma similar:

```
static TextTheme temaTextoOscuro = TextTheme(  
  bodyLarge: GoogleFonts.openSans(  
    fontSize: 14.0,  
    fontWeight: FontWeight.w700,  
    color: Colors.white,  
  ),  
  displayLarge: GoogleFonts.openSans(  
    fontSize: 32.0,  
    fontWeight: FontWeight.bold,  
    color: Colors.white,  
  ),  
);
```

Veamos algunas consideraciones sobre estas propiedades:

- Los temas de texto, son objetos de la clase `TextTheme`
- Básicamente, en un tema de texto, lo que hay que hacer es redefinir para cada estilo predeterminado de Flutter (`bodyLarge` , `displayLarge` , etc.) la combinación de tipo de letra, color, tamaño de la fuente, etc. que se quiera.

Esto se hace asignando a la propiedad que corresponde al estilo que se quiera redefinir, un objeto de tipo `GoogleFonts`

Importante: En versiones anteriores de Flutter, los estilos de texto no se llamaban igual, si ves un ejemplo antiguo de código y no te funciona, muy probablemente es porque estás usando nombres de estilos que están **deprecated**.

A continuación se añaden dos métodos de clase llamados: `claro()` y `oscuro()` que devuelven un objeto de tipo `ThemeData`

El método para devolver el tema claro tiene este aspecto:

```
static ThemeData claro() {
  return ThemeData(
    brightness: Brightness.light,
    // - Otras propiedades
    appBarTheme: const AppBarTheme(
      foregroundColor: Colors.black,
      backgroundColor: Colors.white,
    ),
    // - Otras propiedades
    textTheme: temaTextoClaro,
  );
}
```

Veamos algunas consideraciones sobre este método de clase:

- Devuelve un objeto de tipo `ThemeData` que representa a un tema para una aplicación Flutter. En él se encapsulan todos los aspectos visuales de las vistas de la aplicación (colores, tipos de letra, etc.)
- Comienza usando la propiedad `brightness` que indica si es un tema para cuando el sistema operativo está en modo claro u oscuro.
- A continuación se usan las distintas propiedades para los distintos componentes de la interfaz basada en Material Design. Por ejemplo, en el fragmento de código anterior se usa la propiedad `appBarTheme` que, como su nombre indica permite especificar los colores, etc. para la barra de la aplicación.
- Para finalizar, se usa la propiedad `textTheme` para elegir el tema de texto asociado a este tema. Es uno de los que se hayan definido previamente.

El método `oscuro()` se declara de forma similar.

Añadir soporte para la biblioteca Google Fonts

Si escribimos la clase `MonsterChefTema`, tal y como se ha descrito en la sección anterior, aparecen muchas ocurrencias del error de compilación:

```
Undefined name 'GoogleFonts'.
```

Es normal, es porque se usa una biblioteca llamada **Google Fonts** y nuestra aplicación todavía no tiene soporte para ella. Es el momento de añadir esa dependencia al proyecto.

Las **dependencias** que se usan en una aplicación Flutter se definen en el archivo `pubspec.yaml` que hay en el proyecto. Hay que abrir el archivo y bajo las líneas:

```
dependencies:
  flutter:
    sdk: flutter
```

Hay que añadir, `google_fonts: ^4.0.3` de forma que quede así:

```
dependencies:
  flutter:
    sdk: flutter
  google_fonts: ^4.0.3
```

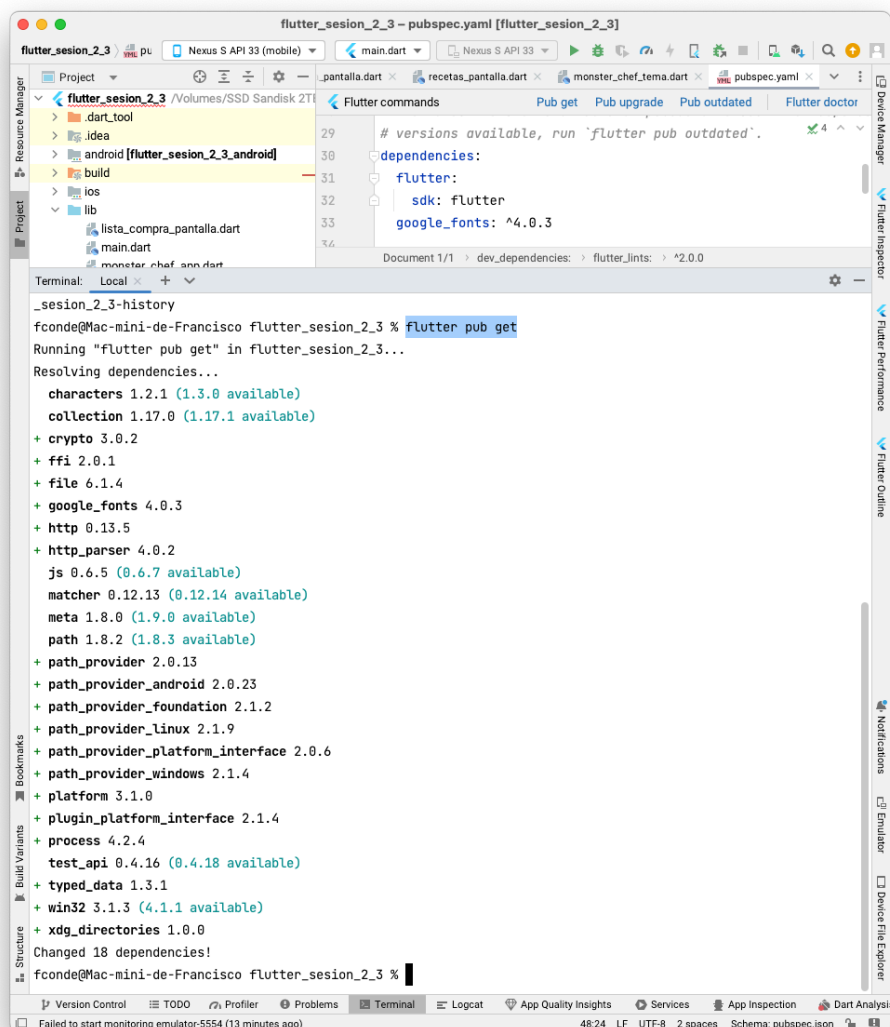
Muy importante: Hay que respetar el sangrado para que funcione correctamente.

Una vez añadido el texto, hay que salvar el archivo con `Ctrl + S` (Windows) o `Cmd + S` (MacOS) y ejecutar:

```
flutter pub get
```

para que se resuelvan las dependencias. Eso puede hacerse de varias formas:

1. En Android Studio, abrir un terminal haciendo clic en la pestaña `Terminal` de la parte inferior y escribir `flutter pub get`



Ejecución de `flutter pub get` desde un Terminal en Android Studio.

2. También en Android Studio, cada vez que se modifica el archivo `pubspec.yaml` aparecen en la parte superior varios enlaces. Uno de ellos es `Pub get`. Si se hace clic sobre él se actualizan las dependencias.



Ejecución de `flutter pub get` desde los propios enlaces que se muestran en la parte superior de Android Studio cuando se modifica el archivo `pubspec.yaml`.

Usar el tema en la aplicación

Una vez añadido el tema personalizado y añadido el soporte para la biblioteca **Google Fonts** ya se puede usar el tema. Para ello solo hay que buscar en la clase `MonsterChefApp` las líneas:

```
theme: ThemeData(  
  primarySwatch: Colors.blue,  
)
```

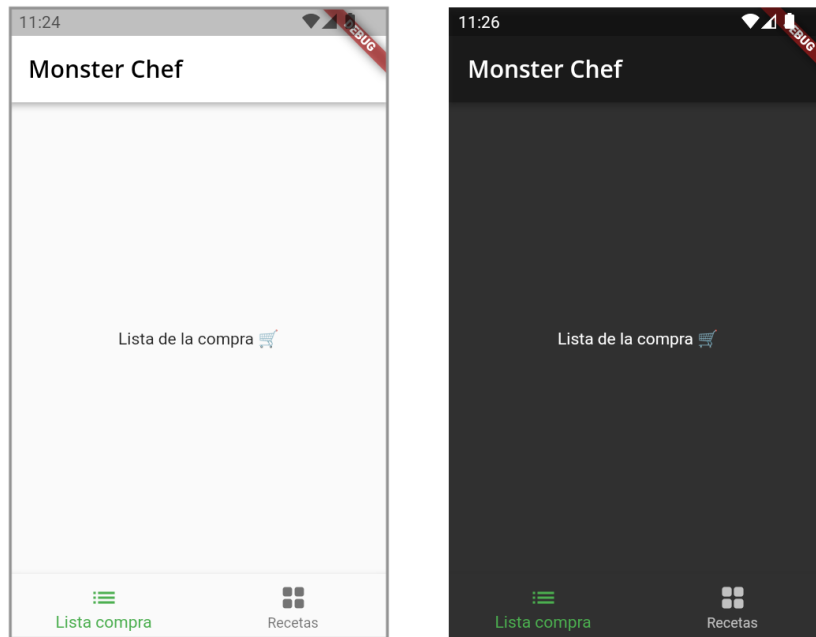
y sustituirlas por estas otras:

```
theme: MonsterChefTema.claro(),  
darkTheme: MonsterChefTema.oscuro(),  
themeMode: ThemeMode.system,
```

Veamos algunas consideraciones interesantes sobre ese código:

- Se usa la propiedad `theme:` de la clase `MaterialApp` para indicar qué tema es el que se usará cuando se elija el modo claro.
- Se usa la propiedad `darkTheme:` de la clase `MaterialApp` para indicar qué tema se usará cuando se elija el modo oscuro.
- Se usa la propiedad `themeMode:` de la clase `MaterialApp` para indicar cómo se seleccionará el modo. En este ejemplo se ha usado la constante `system` del tipo enumerado `ThemeMode` que indica a Flutter que el modo lo tome del que haya definido en el sistema operativo del teléfono. Así, cuando el usuario en su móvil seleccione el tema oscuro, la interfaz de la aplicación Monster Chef cambiará automáticamente a este tema y viceversa.

Este es el aspecto de la interfaz actual en ambos modos:

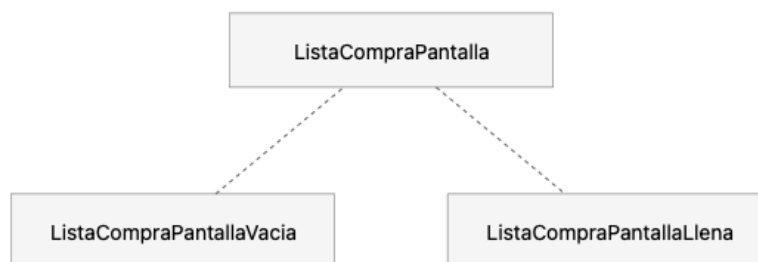


Aspecto de la aplicación Monster Chef en modo claro (izquierda) y oscuro (derecha).

6 – Diseño de las pantallas

La pantalla para la lista de la compra es especial. Debe mostrar una pantalla vacía con indicaciones para el usuario cuando no haya productos en la lista de la compra y una pantalla con la lista de la compra cuando sí haya productos.

La configuración es la siguiente:



Configuración de pantallas para la lista de la compra.

Veamos con mas detalle el diseño de la pantalla `ListaCompraPantalla` :

- Además de la clase `ListaCompraPantalla` habrá otras dos clases `ListaCompraPantallaVacía` que se mostrará cuando no haya elementos en la lista y `ListaCompraPantallaLlena` que se mostrará cuando la lista tenga productos.
- La decisión de cuál de las dos pantalla se muestra depende del estado de la lista de la compra y la toma la clase `ListaCompraPantalla` .

El problema es distinto al que hemos gestionado hasta ahora, donde el estado mutable y la pantalla que refleja los cambios en ese estado estaban dentro de la misma clase.

Ahora, el estado mutable está dentro de la lista de la compra que se usa en varios sitios por tanto no puede formar parte de `ListaCompraPantalla`. Eso requiere otra forma para gestionar el estado.

En Flutter se usa el patrón de diseño de software llamado **estado** para gestionar esta situación. Lo programaremos un poco más adelante en esta sesión de prácticas.

- También hará falta un botón para añadir nuevos productos a la lista de la compra. Este botón se debe mostrar tanto en la pantalla `ListaCompraPantallaVacía` como en la pantalla `ListaCompraPantallaLlena`, y en ambas pantallas se comporta exactamente de la misma forma. No podemos repetirlo o estaríamos incumpliendo el principio DRY, por eso, será un elemento de la clase `ListaCompraPantalla` y las otras dos pantallas se mostrarán dentro de ella, como subelementos de `ListaCompraPantalla`
- También se necesitará una pantalla para añadir los nuevos productos a la lista de la compra, a la que se accederá tanto desde `ListaCompraPantallaVacía` como desde `ListaCompraPantallaLlena` mediante un patrón de navegación **Información anidada**.

Ahora que tenemos claro el diseño, vamos a comenzar a programarlo.

Consejo: Nunca comiences a programar una aplicación sin saber qué es lo que vas a programar, cómo lo vas a programar y por qué.

7 – Pantalla para cuando la lista de la compra está vacía

El primer paso es añadir al proyecto una clase llamada `ListaCompraPantallaVacía` que es la vista que se mostrará cuando no haya productos en la lista de la compra. Es una pantalla que muestra un texto explicativo en el centro con indicaciones para el usuario sobre cómo añadir productos.

Es una subclase de `StatelessWidget` ya que no va a cambiar en toda la ejecución de la aplicación. El texto explicativo será siempre el mismo. El método `build` de esta clase devuelve:

```
return Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      Text(
        'No hay productos todavía',
        style: Theme.of(context).textTheme.titleLarge,
      ),
      const SizedBox(height: 16.0,),
      Text(
        'Pulsa el botón + para añadirlos',
        textAlign: TextAlign.center,
        style: Theme.of(context).textTheme.bodyMedium,
      ),
    ],
  ),
);
```

Veamos algunas consideraciones sobre esta clase:

- El tema personalizado se añadió a la aplicación en el paso 5. Se añadió a la vista `MaterialApp` dentro de la clase `MonsterChefApp`. Esa clase es el origen del árbol de vistas. La clase `ListaCompraPantallaVacía` es un nodo hoja dentro de ese árbol.

Para acceder al tema de la aplicación hay que usar:

```
Theme.of(context)
```

que accede al tema a través del **contexto común compartido** por todas las vistas del árbol de vistas. Ese contexto se pasa a cada vista hija al ejecutar su método `build(BuildContext: context)`

A través de ese tema de la aplicación se puede acceder a todos los ajustes personalizados que se han definido en la clase `MonsterChefTema`. En este caso se han usado las propiedades del tema de texto `titleLarge` para la primera línea de texto y `bodyMedium` para la segunda.

8 – Mostrar la pantalla para cuando la lista de la compra está vacía

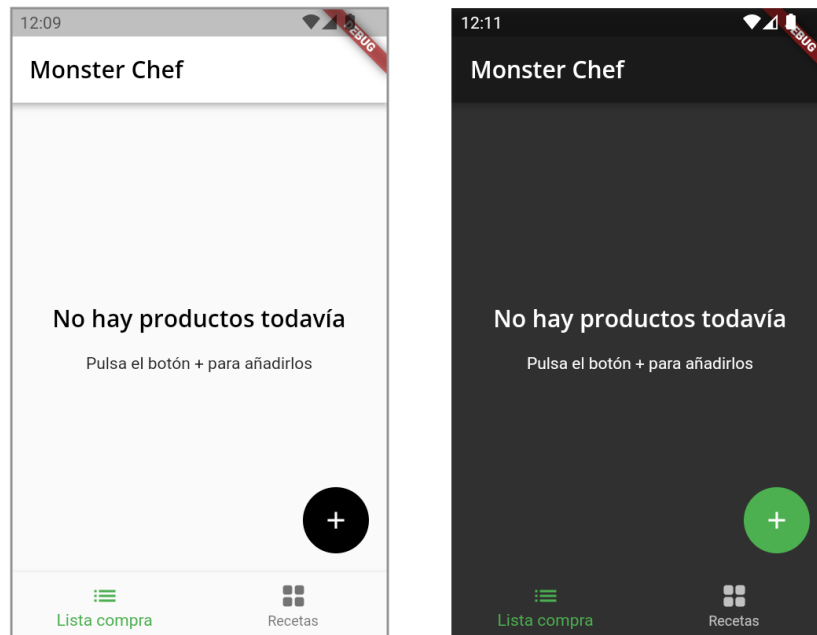
Ya se ha programado la clase `ListaCompraPantallaVacía` ahora es el momento de mostrarla al usuario como un subelemento dentro de la pantalla `ListaCompraPantalla`, por eso, una vez escrito su código, en `ListaCompraPantalla` buscaremos las líneas:

```
return const Center(  
  child: Text('Lista de la compra 🛒'),  
);
```

y las sustituiremos por:

```
return Scaffold(  
  body: const ListaCompraPantallaVacía(),  
  floatingActionButton: FloatingActionButton(  
    elevation: 0,  
    onPressed: () {  
    },  
    child: const Icon(Icons.add),  
  ),  
);
```


Si ejecutamos la aplicación en un emulador se obtiene:



Aspecto de la pantalla para la lista de la compra vacía en los modos claro y oscuro.

Veamos algunos aspectos interesantes de ese código:

- Dentro del método `build` de la vista `ListaCompraPantalla` se devuelve una vista de tipo `Scaffold`. Además, la vista `ListaCompraPantalla` está dentro de la vista `MonsterChefPaginaPrincipal` cuyo método `build` también devuelve una vista de tipo `Scaffold`.

Se tiene una vista `Scaffold` dentro de otra. Flutter lo permite, de forma que se pueda tener una barra de aplicación y una barra de navegación comunes a todas las pantallas, junto con FABs u otros elementos personalizados para algunas de las pantallas. De esta forma se puede construir una interfaz muy flexible sin tener que repetir elementos en varias vistas, lo que ayuda a cumplir el principio DRY.

- Para programar el botón para añadir nuevos productos a la lista de la compra se ha usado un FAB (Floating Action Button). En Flutter se programa a través de una vista `Scaffold` mediante la propiedad `floatingActionButton`. Esta propiedad recibe un objeto de la clase `FloatingActionButton` que tiene multitud de propiedades. Aquí se han usado tres:
 - `child`: que permite definir el contenido del FAB. Lo normal es que se pase un icono por medio de una vista de clase `Icon`.
 - `elevation`: que controla la sombra del FAB sobre la pantalla. El valor por defecto es `6` lo que proyecta una pequeña sombra. En este caso no se quiere sombra para no recargar la interfaz y se ha usado el valor `0`.
 - `onPressed`: que recibe como valor una función anónima. La que realiza la tarea asignada al botón. En este caso está vacía, por eso, el botón no es funcional.

Nota: Añadiremos su funcionalidad un poco más adelante.

9 – El patrón estado en Flutter

Para programar la lista de la compra, que es el modelo de datos de la aplicación, es necesario conocer cómo funciona el patrón estado en Flutter. Esto es así, ya que la lista de la compra es el estado mutable de las vistas `ListaCompraPantalla` y `ListaCompraPantallaLlena` y el paradigma estudiado en la sesión anterior no sirve para modelar este caso.

La lista de la compra es el **estado mutable** de la aplicación, pero a diferencia de lo que ocurría en la sesión anterior de prácticas, en la que los cambios **se producían y consumían** en la misma vista, en esta sesión los cambios se producen en la vista para añadir/modificar productos de la lista y se consumen en las vistas `ListaCompraPantalla` y `ListaCompraPantallaLlena`. Se necesita algún mecanismo para que varias vistas distintas accedan al estado mutable y para que cuando haya cambios, todas sean notificadas para poder refrescarse adecuadamente.

Este mecanismo es el **patrón estado**, que en Flutter se implementa mediante el mecanismo conocido como **provider - consumer**.

Hay cuatro conceptos importantes relacionados con el enfoque provider - consumer:

- `ChangeNotifier` Es la clase que en el patrón estado juega el papel de **Observable**. Es la clase que contiene el estado mutable cuyos cambios se observan desde otras clases que deben refrescarse cuando se produzcan cambios. Cuando en una aplicación se necesite un observable, se debe construir una subclase de `ChangeNotifier`, esta subclase contendrá el estado mutable y métodos para cambiarlo. En esos métodos se debe incluir la llamada al método `notifyListeners()` heredado de la superclase `ChangeNotifier` que notifica a los observadores que se han producido cambios en el observable.
- `ChangeNotifierProvider` Es el **proveedor**. Es quien da acceso al estado mutable a las vistas que lo necesitan. Cuando se crea, en su constructor se debe pasar una función anónima a la propiedad `create`: cuya función es construir el `ChangeNotifier` que va a contener el estado mutable a observar y asociarlo con el contexto de la aplicación. Esto se hace en la vista principal de la aplicación (en este caso `MonsterChefApp`), de forma que el observable estará disponible para cualquier vista de la aplicación que necesite observar los cambios en ese estado.
- `Consumer` Es la clase que en el patrón estado juega el papel de **Observador**. Su tarea es escuchar los cambios que se producen en un `ChangeNotifier`. Si se quiere que un subárbol del árbol de vistas observe los cambios en un observable que encapsula a un estado mutable concreto, hay que incluir el subárbol de vistas dentro de una vista de clase `Consumer`. Cada vez que el estado mutable cambie, el subárbol asociado al consumer será notificado y se refrescará.
- `Provider.of` Permite acceder al estado mutable, pero sin escuchar los cambios en ese estado. Se usa en las vistas que originan los cambios, por ejemplo los botones de aceptar cambios. Esas vistas no reflejan el estado mutable, por tanto no son observadores o `Consumer` pero sí deben tener acceso para, por ejemplo: añadir o eliminar elementos a la lista de la compra.

Puede parecer complejo, pero en realidad no lo es. La mejor forma de terminarlo es verlo en acción.

Para ello, hay que añadir una nueva dependencia al proyecto Flutter. En el archivo `pubspec.yaml` añadiremos justo a continuación de: `google_fonts: ^4.0.3` y con el mismo sangrado, la línea:

```
provider: ^6.0.5
```

Importante: A la hora de escribir las dependencias en el archivo `pubspec.yaml` hay que respetar el sangrado o de lo contrario no funcionará.

Importante: Como siempre que se cambia el archivo `pubspec.yaml` hay que **salvarlo** y ejecutar: `flutter pub get`

10 – Estado mutable para la lista de la compra

Representar un producto

Una lista de la compra es una lista de productos, por eso, necesitamos una clase `Producto` para encapsular el comportamiento de un producto individual de la lista de la compra. También necesitaremos un tipo enumerado `Importancia` para codificar los niveles de importancia del producto en la lista de la compra.

El código de la clase `Producto` y el enumerado `Importancia` es:

```
enum Importancia {  
  baja,  
  media,  
  alta,  
}  
  
class Producto {  
  final String id;  
  final String nombre;  
  final Importancia importancia;  
  final int cantidad;  
  final bool completado;  
  
  Producto({  
    required this.id,  
    required this.nombre,  
    required this.importancia,  
    required this.cantidad,  
    this.completado = false,  
  });  
}
```

(Continúa en la página siguiente)

```

Producto copiaSiNulo({
    String? id,
    String? nombre,
    Importancia? importancia,
    int? cantidad,
    bool? completado,
}) {
    return Producto(
        id: id ?? this.id,
        nombre: nombre ?? this.nombre,
        importancia: importancia ?? this.importancia,
        cantidad: cantidad ?? this.cantidad,
        completado: completado ?? this.completado,
    );
}

```

Veamos algunos aspectos interesantes de esta clase:

- Cada objeto de la clase `Producto` tiene una propiedad llamada `id` para diferenciar unos productos de otros. Poder diferenciarlos es fundamental para mostrarlos correctamente en la interfaz de usuario. Más adelante usaremos el paquete `Uuid` para generar ids únicos de tipo `String` para cada producto.
- Se ha añadido un constructor generativo personalizado para poder construir instancias de la clase `Producto` a partir de los datos que se introduzcan en la interfaz de usuario.

Todos los parámetros del constructor son parámetros con nombre y requeridos, a excepción del parámetro `completado` que se ha dejado opcional. Esto es así porque cuando se da de alta un producto, no está completado y su valor siempre va a ser `false` (no tiene mucho sentido añadir un producto ya completado a una lista de la compra)

Es posible que te preguntes, ¿por qué añadirlo como parámetro al constructor? ¿no sería mejor declarar la propiedad `completado` así?

```
final bool completado = false;
```

¿y de esta forma eliminar el parámetro en el constructor?

Pista: Mira el resto de código y enseguida descubrirás que no es posible.

- También hay un método de instancia llamado `copiaSiNulo` que funciona como un constructor de copia, pero mucho más potente y flexible.

Recibe un parámetro opcional, con nombre y con capacidad de tener valores nulos, para cada propiedad de la clase.

Es un método de instancia, por tanto, solo se puede llamar a través de un objeto de la clase `Producto`. Cuando se llama al método `copiaSiNulo` sobre un objeto de la clase `Producto`, si no se pasa nada en un parámetro (se tiene un valor nulo) se toma el valor de propiedad del objeto, y si se pasa algún valor, se toma como valor de la propiedad ese valor.

Por ejemplo, si se ha construido el siguiente objeto:

```
final producto = Producto(  
  id: 'abc',  
  nombre: 'Manzanas',  
  importancia: Importancia.media,  
  cantidad: 4  
);
```

y a continuación se escribe:

```
final productoCopiado = producto.copiaSiNulo(importancia: Importancia.baja);
```

el objeto resultado de la copia: `productoCopiado`, tendrá como valores de propiedad:

```
id: 'abc'  
nombre: 'Manzanas'  
importancia: Importancia.baja  
cantidad: 4
```

Es decir, es una copia del objeto `producto` que tiene los mismos valores de propiedad, excepto en la propiedad `importancia` para la que se ha pasado un valor distinto como argumento del método `copiaSiNulo`

Este método es muy útil para modificar los valores de algunas propiedades de un `Producto` dejando las demás intactas. Lo usaremos para modificar productos de la lista de la compra.

Representar la lista de la compra

Añadiremos un nuevo archivo al proyecto llamado `lista_compra.dart` y en él escribiremos la clase para gestionar el estado mutable de esta parte de la aplicación. Como sabemos es un Observable que en Flutter se programa como una subclase de `ChangeNotifier`. El código de la clase `ListaCompra` queda así:

```
class ListaCompra extends ChangeNotifier {  
  final _productos = <Producto>[];  
  List<Producto> get productos => List.unmodifiable(_productos);  
  
  void borraProducto(int indice) {  
    _productos.removeAt(indice);  
    notifyListeners();  
  }  
  
  void anadeProducto(Producto item) {  
    _productos.add(item);  
    notifyListeners();  
  }  
}
```

(Continúa en la página siguiente)

```

void actualizaProducto(Producto item, int indice) {
    _productos[indice] = item;
    notifyListeners();
}

void marcaCompletado(int indice, bool completado) {
    final producto = _productos[indice];
    _productos[indice] = producto.copiaSiNulo(completado: completado);
    notifyListeners();
}
}

```

Veamos algunas consideraciones sobre este código:

- `ListaCompra` es un observable, contiene un estado mutable cuyos cambios se quieren observar desde otras vistas, por tanto, debe ser una subclase de `ChangeNotifier`
- Tiene una propiedad privada llamada `_productos` que contiene la lista de la compra. Esa propiedad es el estado mutable encapsulado en este `ChangeNotifier`
- Tiene un getter llamado `productos` para permitir que la lista pueda ser consultada desde otras vistas que deban refrescarse en función del contenido de la lista de la compra. Para que no se puedan hacer cambios no deseados o no controlados en la lista, el getter proporciona una copia inmutable mediante `List.unmodifiable(_productos)`; De esta forma desde otras clases no se puede cambiar su longitud (añadir o borrar elementos), ni modificar los elementos existentes.
- Se incluyen cuatro métodos para modificar la lista de la compra: `borraProducto(int indice)`, `anadeProducto(Producto producto)`, `actualizaProducto(Producto item, int indice)` y `marcaCompletado(int indice, bool completado)`. En todos ellos se tiene como última instrucción `notifyListeners()`; que notifica a los observadores registrados que se han producido los cambios y por tanto deben refrescarse para mostrarlos.

Muy importante: En todos los métodos que modifican el estado mutable almacenado en un observable `ChangeNotifier` la última línea debe ser `notifyListeners()`; . De lo contrario, los observadores no sabrán que se han producido cambios y no se refrescarán.

11 – Proveedor de acceso a la listad de la compra

Ya que se tiene el observable que encapsula la lista de la compra, la clase `ListaCompra`, es el momento de hacerlo disponible para todas las vistas de la aplicación que quieran registrarse para ser notificadas cuando haya cambios. Eso se hace mediante un provider.

Para proporcionar acceso al `ChangeNotifier` abriremos el archivo: `monster_chef_app.dart`, buscaremos la línea:

```

home: MonsterChefPaginaPrincipal(titulo: 'Monster Chef',),

```

y la sustituiremos por las líneas:

```
home: MultiProvider(  
  providers: [  
    ChangeNotifierProvider( create: (context) => ListaCompra(),),  
  ],  
  child: MonsterChefPaginaPrincipal(titulo: 'Monster Chef',),  
)
```

Para tener acceso a `MultiProvider` y `ChangeNotifierProvider` es necesario el import:

```
import 'package:provider/provider.dart';
```

Algunas consideraciones sobre el código anterior:

- En la propiedad `home:` que contiene la vista principal de la aplicación, se añade el provider que encapsula al observable `ListaCompra` y lo hace disponible para todo el árbol de vistas de la aplicación.

Para ello se usa la vista `MultiProvider` que tiene como propiedades una lista de `ChangeNotifierProvider` llamada `providers:` y el árbol de vistas de la aplicación `child:`

Se usa `MultiProvider` porque lo normal es que en una aplicación se observe a más de un observable, cada uno de ellos subclase de `ChangeNotifier` y encapsulado dentro de una clase `ChangeNotifierProvider`. En esta aplicación solo se tiene un observable y por eso la lista solo contiene un elemento.

- La función anónima que se pasa en la propiedad `create:` del constructor de la clase `ChangeNotifierProvider` tiene como argumento el contexto de la aplicación, ya que los observables están disponibles para los observadores a través del contexto.
- Los `ChangeNotifier` se crean justo en el momento de hacerlos disponibles para el árbol de vistas a través de un `ChangeNotifierProvider` aunque también se podrían haber creado antes y aquí pasar la variable que contiene al `ChangeNotifier`

Con esto, cada vista del árbol de vistas de la aplicación tiene acceso al observable `ListaCompra` que encapsula a la lista de la compra, y si lo necesita se puede registrar como observador `Consumer` de ese observable.

12 – Consumir los cambios

Los cambios que se producen en un observable se consumen en un observador que en el caso de Flutter se implementa mediante la clase `Consumer`. Si en una determinada vista o subárbol de vistas se necesita estar informado de los cambios en un observable, esa vista o subárbol debe incluirse dentro de una vista de la clase `Consumer`

En la aplicación Monster Chef, los cambios en la lista de la compra se consumen en diferentes parte de la aplicación. Una de ellas es en la vista `ListaCompraPantalla` que debe mostrar una pantalla de tipo `ListaCompraPantallaVacía` o de tipo `ListaCompraPantallaLlena` en función de si la lista está vacía o no.

Para hacer que la vista `ListaCompraPantalla` sea un observador, abriremos `lista_compra_pantalla.dart` y al final de la clase `ListaCompraPantalla` añadiremos el siguiente método de utilidad:

```
Widget construirPantallaListaCompra() {  
  return Consumer<ListaCompra>(  
    builder: (context, manager, child) {  
      if (manager.productos.isNotEmpty) {  
        return Container( color: Colors.blueGrey, );  
      } else {  
        return const ListaCompraPantallaVacía();  
      }  
    },  
  );  
}
```

Luego, en el método `build` buscaremos la línea:

```
body: const ListaCompraPantallaVacía(),
```

Y la sustituiremos por la línea:

```
body: construirPantallaListaCompra(),
```

Veamos algunos aspectos interesantes del código anterior:

- La vista devuelta por el método de utilidad `construirPantallaListaCompra` es de clase `Consumer` que debe estar asociada a un `ChangeNotifier` concreto de los que se hayan puesto a disposición de la aplicación mediante la vista `MultiProvider`. En este caso es un observador de `ListaCompra`, por eso la vista que se devuelve es de tipo: `Consumer<ListaCompra>`
- Lo anterior hace que cada vez que se realice un cambio en la lista de la compra se reconstruyan las vistas que se construyen dentro de la propiedad `builder:` del `Consumer`.

Importante: Para ganar eficiencia, solo se deberían incluir dentro de un `Consumer` las vistas que lo necesiten. Si se incluyen vistas que no reflejan de ninguna forma los cambios en un observable se van a reconstruir en vano, ya que su apariencia o contenido no va a cambiar.

- La lista de parámetros que hay que escribir en la función anónima que se pasa a la propiedad `builder:` de una vista `Consumer` (un observador) son tres:
 - `context` que es el contexto de la aplicación.
 - `manager` que es el `ChangeNotifier` al que se quiere observar.
 - `child` permite ahorrar tiempo si alguna de las vistas del subárbol de vistas no debe reconstruirse por los cambios del observable. En ese caso, junto a la propiedad `builder:` del `Consumer` se usa la propiedad `child:` en la que se construye la parte que no cambia. el argumento `child` en la función anónima de la propiedad `builder:` hace referencia a la vista que no cambia. En este ejemplo no hay ninguna parte que no cambie.

- En `ListaCompraPantalla` se debe mostrar la pantalla correspondiente a la situación en la que la lista de la compra está vacía `ListaCompraPantallaVacía` o la pantalla que muestra la lista de productos a comprar cuando la lista no está vacía. Esta última pantalla aún no se ha escrito. Para comprobar si la lista está vacía o no se usa el `if` :

```
if (manager.productos.isNotEmpty)
```

`manager` hace referencia al observable, que en este caso es de clase `ListaCompra` . El tipo se infiere del hecho de que el consumer está asociado a esta clase mediante `Consumer<ListaCompra>` . una de las propiedades de `ListaCompra` es el getter `productos` que da acceso a una copia inmutable de la lista de la compra. Cuando no está vacía, `isNotEmpty` devuelve `true` se devuelve la pantalla para cuando hay elementos en la lista (por ahora un `Container` vacío en color gris azulado), y cuando devuelve `false` se devuelve la pantalla `ListaCompraPantallaVacía`

En este punto ya se tiene el estado mutable (la lista de la compra), encapsulado dentro de un `ChangeNotifier` (observable), puesto a disposición del árbol de vistas mediante un `ChangeNotifierProvider` y consumido (observado) por la vista `ListaCompraPantalla` que es un `Consumer` (observador) Es decir, está todo listo para mostrar pantallas distintas en función de si la lista de la compra está vacía o contiene productos. Para poder probarlo primero es necesario poder añadir productos a la lista de la compra.

13 – Pantalla para añadir productos a la lista de la compra

Lo siguiente es construir la pantalla que aparece cuando se pulsa el FAB `+` para añadir elementos. Es la pantalla para añadir productos a la lista de la compra.

Nota: También usaremos esta pantalla para editar productos ya añadidos a la lista.

Crearemos un nuevo archivo dentro del proyecto al que llamaremos

`lista_compra_anadir_producto.dart` y en él escribiremos la clase `ListaCompraAnadirProducto` que es subclase de `StatefulWidget` :

```
class ListaCompraAnadirProducto extends StatefulWidget {
  final Function(Producto) crearProducto;
  final Function(Producto) editarProducto;
  final Producto? productoOriginal;
  final bool actualizando;

  const ListaCompraAnadirProducto({
    Key? key,
    required this.crearProducto,
    required this.editarProducto,
    this.productoOriginal,
  }) : actualizando = (productoOriginal != null),
        super(key: key);

  @override
  _ListaCompraAnadirProductoState createState() => _ListaCompraAnadirProductoState();
}
```

Algunos detalles importantes de esta clase:

- La pantalla `ListaCompraAnadirProducto` se emplea tanto para añadir nuevos productos a la lista de la compra, como para editar los ya existentes. La interfaz es la misma, pero el proceso varía por lo que hay que poder indicar a la clase si se está añadiendo o editando un producto.

Si se está editando, se le pasa como parámetro el producto que se quiere editar. Para ello se usa la propiedad `productoOriginal`. Si se está añadiendo, se pasa el valor `null`.

Por comodidad se usa una propiedad de tipo `bool` llamada `actualizando` que se inicializa en la lista de inicialización del constructor mediante la expresión:

```
actualizando = (productoOriginal != null)
```

- Hay dos propiedades `crearProducto` y `editarProducto` que almacenan funciones anónimas con un único parámetro de tipo `Producto` definidas como:

```
final Function(Producto) crearProducto;  
final Function(Producto) editarProducto;
```

Son los callback que se ejecutarán cuando el usuario pulse el botón de aceptar  (todavía no se ha construido, eso se hace en el estado asociado). Se pasan desde fuera, en vez de escribir el código de esas funciones anónimas en esta clase, ya que eso permite una mayor flexibilidad.

Como sabemos, toda `StatefulWidget` debe tener su correspondiente clase privada de tipo `State` asociada, para gestionar su estado mutable. Es en esta clase donde se construye la interfaz de usuario:

```
class _ListaCompraAnadirProductoState extends State<ListaCompraAnadirProducto> {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        actions: [  
          IconButton(  
            icon: const Icon(Icons.check),  
            onPressed: () {  
              Navigator.pop(context);  
            },  
          ),  
        ],  
        elevation: 0.0,  
        title: const Text('Añadir/editar', ),  
      ),  
      body: Container(color: Colors.grey),  
    );  
  }  
}
```

Como puede verse, esta clase todavía no está terminada. En próximas secciones se completará. Tiene:

- Una vista de tipo `Scaffold` en la que se ha añadido en la barra de la aplicación `AppBar` no sólo un texto como en otras sesiones, sino que también se ha añadido un botón. Para ello se ha usado la propiedad `actions:` de `AppBar` que acepta una lista de botones con las acciones que se quieran mostrar en la barra de la aplicación.

En esta aplicación sólo se necesita un botón cuyo icono es un tick mark  que permitirá aceptar, bien el nuevo producto para la lista, o bien los cambios en un producto ya existente.

Muy importante: Esta pantalla no se va a insertar en el árbol de vistas de la aplicación, sino que cuando se active, lo va a sustituir. Por eso, hay que escribir la vista `Scaffold` completa. Se va a usar el patrón **Información anidada**. En este patrón, la vista de detalles (la pantalla para editar productos) sustituye a la vista general (la lista de la compra), por tanto **no es un subelemento de la lista de la compra**.

- La propiedad `onPressed:` del botón que es la única acción que se necesita en esta vista, llama al método `pop` de la clase `Navigator` que quita de la pantalla del dispositivo la pantalla para editar productos y vuelve a poner en su lugar la pantalla para mostrar la lista de la compra.

En Flutter, el patrón de Información anidada se implementa mediante las clases `Navigator` y `MaterialPageRoute`

14 – Mostrar la pantalla para añadir/editar productos

La pantalla para añadir/editar productos está incompleta, pero para poder probarla mientras la completamos, merece la pena que nos detengamos a programar el FAB  que hace que se muestre. Así podremos ver y probar los distintos cambios que le hagamos a medida que la vayamos completando.

Desde la pantalla `ListaCompraPantalla`, se muestra la pantalla `ListaCompraAnadirProducto` cuando el usuario pulsa el botón . Ese botón tiene asociado un callback que es una función anónima que se escribe en la propiedad `onPressed:`. Por ahora estaba vacío.

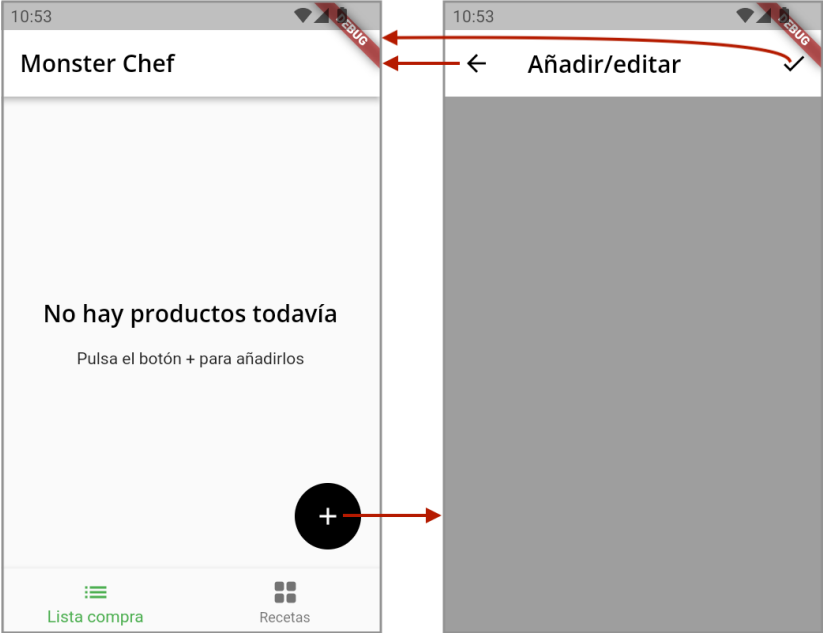
Para que al pulsar el botón  en `ListaCompraPantalla` se muestre la pantalla `ListaCompraAnadirProducto` hay que buscar en el código de la clase `ListaCompraPantalla` las líneas:

```
onPressed:  {  
  },
```

y sustituirlas por las líneas:

```
onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) {
        return ListaCompraAnadirProducto(
          crearProducto: (producto) { Navigator.pop(context); },
          editarProducto: (producto) { },
        );
      },
    ),
  );
},
);
},
);
```

Si aplicamos los cambios, el resultado es:



Botón FAB para mostrar la pantalla de añadir/editar productos completamente funcional. Para volver a la pantalla de lista de la compra se puede pulsar la flecha atrás, lo que cancela la adición/edición, o el tick marck  que acepta los cambios.

Nota: A la izquierda de la barra de la aplicación aparece una flecha para volver a la pantalla anterior. Esta flecha la incluye Flutter automáticamente ya que se han usado las clases `Navigator` y `MaterialPageRoute`. Es la forma de implementar en Flutter el patrón de interacción **Botón atrás**.

Veamos algunas consideraciones sobre el código anterior:

- La propiedad `onPressed:` de la vista `FloatingActionButton` permite asignar la función anónima que se ejecutará como **callback** cuando el usuario pulse el FAB etiquetado como `+`. La función anónima no debe tener parámetros. Mediante esta función indicamos a Flutter cómo queremos que se comporte la aplicación al pulsar.

Se debe pasar de la pantalla de lista de la compra a la pantalla de añadir/editar productos, pero para que el usuario comprenda mejor el cambio, programaremos la transición entre pantallas con una animación.

Para que la transición entre pantallas se realice con una animación se usan dos clases:

- `Navigator` que es un widget que gestiona un conjunto de widgets secundarios que forman una pila. En la pila puede haber varios niveles, aunque en este caso sólo tenemos dos: la pantalla con la lista de la compra y la pantalla para añadir/editar un producto.

Implementa el patrón de interacción llamado **Información anidada**, permite al usuario hacer la transición visual de una pantalla a otra, por lo que es muy intuitivo y sencillo de utilizar.

- `MaterialPageRoute` que permite implementar una transición entre pantallas, con una animación adaptable a la plataforma en la que se va a ejecutar.
 - En el caso de Android, la transición de entrada consiste en deslizar hacia arriba la pantalla nueva y desvanecer la pantalla actual. La transición de salida es la misma, pero a la inversa.
 - En el caso de iOS, la transición de entrada desliza los elementos de la pantalla nueva desde la derecha al mismo tiempo que se deslizan los de la pantalla actual. La transición de salida es la misma pero a la inversa.

Si usamos Flutter no nos tenemos que preocupar por estos detalles. La propia plataforma elegirá la animación correcta en función del sistema operativo del terminal en el que se ejecute la aplicación.

- Por ahora, la función anónima que se pasa en la propiedad `crearProducto:` al constructor de `ListaCompraAnadirProducto` solo contiene la orden `Navigator.pop(context);` que produce que se vuelva a la pantalla `ListaCompraPantalla`. Más adelante se completará.
- La función anónima que se pasa en la propiedad `editarProducto:` al constructor de `ListaCompraAnadirProducto` está vacío ya que si se muestra la pantalla `ListaCompraAnadirProducto` a consecuencia de haber pulsado el botón `+` de la interfaz, es porque se está creando un nuevo producto, no editando uno existente, así que el callback `editarProducto:` nunca se va a ejecutar en esta situación.

15 – Completar la pantalla para añadir productos

Por ahora la pantalla `ListaCompraAnadirProducto` está vacía. Sólo se muestra el fondo en un color gris más oscuro. Es el momento de añadirle las vistas que se van a necesitar para dar valores a las propiedades de la clase `ListaCompra`

Nota: Para hacer más legible el código de la clase `_ListaCompraAnadirProductoState` donde se construyen los distintos elementos de la pantalla para añadir/editar productos de la lista de la compra, el código que construye cada uno de los elementos se encapsulará dentro de un método de utilidad de la propia clase `_ListaCompraAnadirProductoState`. De esta forma es más sencillo de leer y entender.

Añadir el campo texto para el nombre

La primera vista es un campo de texto `TextField` donde el usuario pueda escribir el nombre del producto que quiere añadir a la lista de la compra.

En la sesión anterior ya usamos una vista de esta clase, sin embargo, en esta vamos a gestionarlo con una clase de tipo `TextEditingController` que es la forma más flexible de gestionar una vista de clase `TextField` en Flutter. Este método es un poco más complejo de escribir que el que se usó en la sesión anterior pero ofrece una mayor flexibilidad.

Comenzaremos añadiendo en la clase `_ListaCompraAnadirProductoState` las siguientes dos propiedades:

```
final _controladorNombre = TextEditingController();
String _nombre = '';
```

Veamos algunas cosas interesantes sobre esas dos propiedades:

- `_controladorNombre` es el `TextEditingController` que se usará para gestionar los cambios que el usuario haga al contenido del campo de texto para el nombre del producto.
- `_nombre` almacena el valor que el usuario da al nombre del producto. Su valor inicial es la cadena vacía.

Importante: La propiedad `_nombre` forma parte del estado mutable de la clase, por tanto, siempre que se actualice debe hacerse dentro de una función anónima que se pase al método `setState` de la clase `State`

Nota: Mas adelante, cuando se escriba la pantalla para mostrar la lista de la compra, y se pueda editar alguno de los productos que contiene, se podrá cambiar para el caso de que no se está añadiendo un nuevo producto, sino editando uno ya creado. En ese caso, el valor inicial de `_name` no será la cadena vacía sino el nombre del producto que se esté editando.

A continuación sobreescribiremos los métodos `initState` y `dispose` de la clase `_ListaCompraAnadirProductoState` con el siguiente contenido:

```
@override
void initState() {
  super.initState();

  _controladorNombre.addListener(() {
    setState(() { _nombre = _controladorNombre.text; });
  });
}

@override
void dispose() {
  _controladorNombre.dispose();
  super.dispose();
}
```

Veamos con detalle este nuevo código:

- Todas las vistas tienen el método `initState` que se ejecuta antes de que se construya la vista. Se puede utilizar para inicializar algunos aspectos de las propiedades de las vistas antes de mostrarlas en la pantalla.

En este caso se usa para añadir a la clase `_ListaCompraAnadirProductoState` como observadora del `TextEditingController` llamado `_controladorNombre` que se usa para controlar los cambios en el `TextField`. Siempre que el usuario cambie el contenido del campo de texto para el nombre del producto, se llamará a la función anónima que se pasa a `addListener`. En este caso se actualiza el valor de la propiedad `_nombre`.

- También tienen el método `dispose` que se ejecuta **cuando la vista se elimina del árbol de vistas**. Se usa para liberar recursos, etc. En este caso, se usa para eliminar el `TextEditingController` llamado `_controladorNombre` que ya no será necesario cuando la vista se elimine del árbol de vistas.

A continuación se añadirá a la clase `_ListaCompraAnadirProductoState` el siguiente método:

```
Widget construyeCampoNombre() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: <Widget>[
      Text(
        'Nombre del producto:',
        style: Theme.of(context).textTheme.titleSmall,
      ),
      TextField(
        controller: _controladorNombre,
        decoration: const InputDecoration(
          hintText: 'P.e.: Pan, 1kg de sal, etc.',
        ),
      ),
    ],
  );
}
```

Veamos algunos detalles del código recién añadido:

- Añade dos vistas, una con un texto explicativo y otra con el campo de texto editable. El proceso es similar al seguido en la sesión anterior cuando se construyó el conversor. Se usa una vista de clase `Column` para situar ambos elementos uno sobre otro.

En esta ocasión se ha usado el valor `CrossAxisAlignment.start` para la propiedad `crossAxisAlignment:` de la clase `Column` para que los textos se alineen a la izquierda.

- En la propiedad `controller:` del `TextField` se asigna el `TextEditingController` que queremos que gestione los cambios en el texto, en este caso `_controladorNombre`.
- En la propiedad `decoration:` de tipo `InputDecoration` hay una propiedad llamada `hintText:` en la que se puede escribir el texto que queremos que se muestre en el campo cuando esté vacío.

Lo normal es que se escriba alguna pista sobre lo que hay que escribir en el campo. Eso mejora mucho la experiencia de uso.

Y por último, para que realmente se muestre el nuevo campo de texto en la interfaz, buscaremos en el método `build` de la clase `_ListaCompraAnadirProductoState` la línea:

```
body: Container(color: Colors.grey),
```

y la sustituiremos por:

```
body: Container(  
  padding: const EdgeInsets.all(16.0),  
  child: ListView(  
    children: <Widget>[  
      construyeCampoNombre(),  
    ],  
  ),  
)
```

Veamos con más detalle este código:

- Se usa una vista de tipo `ListView` para organizar las propiedades de un `Producto` de forma que se pueda hacer scroll. Esto permite seguir viendo todas las propiedades aunque el tamaño de pantalla de un dispositivo concreto sea pequeño y no quepan todas a la vez.

Nota: La vista `ListView` de Flutter permite el desplazamiento de su contenido (scroll). Contiene una lista de widgets organizados linealmente. Es ideal para representar listas de objetos en la interfaz de usuario.

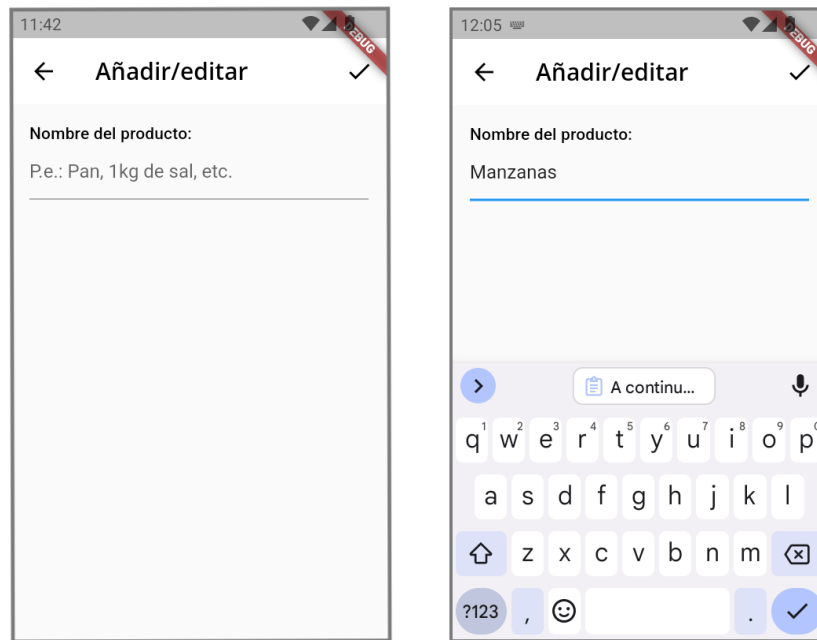
- Dado que, en este caso, no todos los elementos del `ListView` son del mismo tipo, se usará el constructor parametrizado de `ListView` que espera la propiedad `children:` que es una lista con las vistas que se quieren mostrar dentro del `ListView`

Por ahora, solo se tiene la vista que devuelve el método de utilidad `buildNameField` pero en breve se completará con las demás.

- Para mejorar la apariencia, se ha encerrado el `ListView` dentro de una vista de tipo `Padding` que añade espacio alrededor. De esta forma los datos del producto que se añade/edita no llegan justo hasta el borde, lo que añade legibilidad.

Si hacemos hot reload, veremos que ahora la pantalla para añadir un nuevo producto no está vacía, sino que contiene el campo de texto para añadir el nombre del producto.

El aspecto de la aplicación es el siguiente:



Aspecto de la pantalla para añadir/editar productos tras añadirle el campo para escribir el nombre del producto. A la izquierda el campo vacío sin el foco por lo que se muestra la pista sobre cómo rellenar el campo. A la derecha el campo con información y con el foco por lo que la pista ha desaparecido sustituida por el valor tecleado y también se muestra el teclado.

Añadir vistas para seleccionar la importancia del producto

Una de las propiedades de la clase `Producto` es `importancia` que es del tipo enumerado `Importancia` que permite codificar si el producto tiene importancia alta, media o baja dentro de la lista de la compra.

A continuación vamos a añadir a la pantalla para añadir/editar productos las vistas que permiten codificar esa importancia. Se ha optado por usar la clase `ChoiceChip` de Flutter, que tiene el mismo comportamiento que un radio button, pero con otra apariencia visual.

Comenzaremos añadiendo una nueva propiedad a la clase `_ListaCompraAnadirProductoState`

```
Importancia _importancia = Importancia.baja;
```

Importante: La propiedad `_importancia` forma parte del estado mutable de la clase, por tanto, siempre que se actualice debe hacerse dentro de una función anónima que se pase al método `setState` de la clase `State`

A continuación modificaremos ligeramente el tipo enumerado llamado `Importancia` añadiéndole una propiedad llamada `_nombre` con su correspondiente getter. Eso nos permitirá rellenar los chips de forma automática.

```
enum Importancia {
  baja('baja'),
  media('media'),
  alta('alta');

  final String _nombre;
  String get nombre => _nombre;
  const Importancia(this._nombre);
}
```

A continuación añadiremos otro método de utilidad en `_ListaCompraAnadirProductoState` llamado `construyeCampoImportancia` con el siguiente código:

```
Widget construyeCampoImportancia() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: <Widget>[
      Text(
        'Importancia',
        style: Theme.of(context).textTheme.titleSmall,
      ),
      Wrap(
        spacing: 10.0,
        children: Importancia.values.map(
          (elemento) {
            return ChoiceChip(
              selectedColor: Colors.black,
              selected: _importancia == elemento,
              label: Text(
                elemento.nombre,
                style: TextStyle(color: Colors.white),
              ),
              onPressed: (seleccion) {
                setState(() { _importancia = elemento; });
              },
            );
          }
        ).toList(),
      ),
    ],
  );
}
```

Veamos algunas consideraciones sobre este método de utilidad:

- Para almacenar los `ChoiceChip` en la pantalla se usa la vista `Wrap` que recibe en su propiedad `children`: una lista de vistas y las muestra en una fila mientras haya sitio. Si se termina el espacio horizontal disponible, las demás vistas se muestran en filas sucesivas.

Nota: Es una vista muy útil cuando se deben mostrar organizadas por filas una lista de vistas y queremos asegurar que se ven bien con independencia del tamaño de la pantalla del smartphone concreto que se use.

- En cada `ChoiceChip` hay que pasar una función anónima a la propiedad `onSelected`: Esa función anónima indica qué debe hacerse cuando el usuario selecciona ese chip. En este caso hay que actualizar la propiedad `_importancia` de la clase `_ListaCompraAnadirProductoState`

- Los chips se rellenan automáticamente. Para eso se usa el higher-order method `map` aplicado al iterable con todos los valores del tipo enumerado `Importancia` que devuelve su propiedad `values`

Recuerda que `map` espera un único argumento (en este ejemplo lo he llamado `elemento`) cuyo tipo se infiere al tipo de los elementos del iterable sobre el que se aplica. En este caso son las constantes de tipo `Importancia`

Para cada elemento se construye un `ChoiceChip` usando el getter que se ha añadido un poco más arriba al tipo enumerado `Importancia` para obtener una cadena de caracteres con el nombre de la constante del tipo enumerado que se tenga en `elemento`, y pasar ese valor como etiqueta del chip en su propiedad `label`:

Por último, dado que `map` devuelve un `Iterable` y `children` espera una `List<Widget>` hay que llamar al método `toList()` de los iterables para hacer la conversión.

Fíjate bien: con esta forma de rellenar los `ChoiceChip` si se quiere añadir una nueva constante para la importancia, por ejemplo `muy_importante` los únicos cambios que hay que hacer se realizan en el tipo enumerado. La interfaz funcionará correctamente sin tener que modificar nada. Se cumple el principio de diseño de software OCP.

- La propiedad `selected` toma un valor de tipo `bool`. Si tiene valor `true` el chip se dibuja seleccionado (color más oscuro) y si tiene valor `false` se dibuja no seleccionado (color más claro).

En el chip que corresponda con el valor inicial, se debe pasar el valor `true` a la propiedad `selected` y `false` en los demás. Para desacoplar el código no se escribe directamente el valor `true` o `false` sino que se hace la comparación:

```
_importancia == elemento,
```

De esta forma si se quiere cambiar el valor inicial para la propiedad `_importancia` de la clase `_ListaCompraAnadirProductoState`, no hay que reescribir el código que inicializa los chips. De nuevo, en este caso se cumple el principio OCP.

Por último hay que mostrar las vistas anteriores en la pantalla `ListaCompraAnadirProducto`. Para ello, en el método `build` de la clase `_ListaCompraAnadirProductoState`, justo a continuación de:

```
construyeCampoNombre(),
```

hay que añadir:

```
const SizedBox(height: 14,),  
construyeCampoImportancia(),
```

Si hacemos hot reload, el aspecto de la pantalla para añadir/editar productos es:



Aspecto de la interfaz de usuario tras añadir el campo importancia.

Añadir el selector para la cantidad

Por último, se necesita un modo para que el usuario indique la cantidad de producto que hay que comprar. Se ha optado por usar una vista `Slider`.

Comenzaremos añadiendo una nueva propiedad a la clase `_ListaCompraAnadirProductoState`

```
int _valorActualSlider = 0;
```

Importante: La propiedad `_valorActualSlider` forma parte del estado mutable de la clase, por tanto, siempre que se actualice debe hacerse dentro de una función anónima que se pase al método `setState` de la clase `State`

A continuación añadiremos un último método de utilidad a la clase

`_ListaCompraAnadirProductoState`

```
Widget construyeCampoCantidad() {  
  return Column(  
    crossAxisAlignment: CrossAxisAlignment.start,  
    children: <Widget>[  
      Row(  
        crossAxisAlignment: CrossAxisAlignment.baseline,  
        textBaseline: TextBaseline.alphabetic,  
        children: <Widget>[  
          Text(  
            'Cantidad',  
            style: Theme.of(context).textTheme.titleSmall,  
          ),  
        ],  
      ),  
    ],  
  );  
}
```

(Continúa en la página siguiente)

```

        const SizedBox(width: 16.0),
        Text(
          _valorActualSlider.toInt().toString(),
          style: Theme.of(context).textTheme.headline6,
        ),
      ],
    ),
    Slider(
      value: _valorActualSlider.toDouble(),
      min: 0.0,
      max: 100.0,
      divisions: 100,
      label: _valorActualSlider.toInt().toString(),
      onChanged: (double value) {
        setState( () { _valorActualSlider = value.toInt(); },);
      },
    ),
  ],
);
}

```

Veamos algunas consideraciones sobre este código:

- La propiedad `_valorActualSlider` es de tipo `int`, pero la vista `Slider` usa valores de tipo `double`, por tanto es necesario hacer las siguientes conversiones:

```
value: _valorActualSlider.toDouble(),
```

```
_valorActualSlider = value.toInt();
```

- Una de las propiedades del `Slider` es `onChanged`: A esta propiedad se le pasa una función anónima que es el callback que se invoca cuando el usuario desliza el `Slider` y cambia su valor.

En esa función anónima se debe cambiar el valor de `_valorActualSlider` pero como forma parte del estado mutable de la pantalla para añadir/editar productos, se debe hacer la asignación dentro de una llamada al método `setState` de la clase `State`

- Además del `Slider` para cambiar el valor de la cantidad de producto, se usa una vista de tipo `Text` para que el usuario pueda ver claramente la cantidad seleccionada.

Esa vista muestra el valor de la propiedad `_valorActualSlider` que es parte del estado mutable de la vista `_ListaCompraAnadirProductoState`, por tanto, cada vez que se cambia por medio del `Slider`, se reconstruye la pantalla para mostrar el nuevo valor.

- Junto a la vista `Text` para mostrar el valor del `Slider` se muestra otra vista `Text` con una etiqueta que indica que el valor que visualiza a continuación es la cantidad de producto.

Ambas vistas deben figurar una al lado de la otra, por lo que se ha usado una vista de clase `Row` para agruparlas. Es idéntica a la vista de clase `Column` que ya hemos usado, pero coloca sus vistas hijas en fila en lugar de en columna.

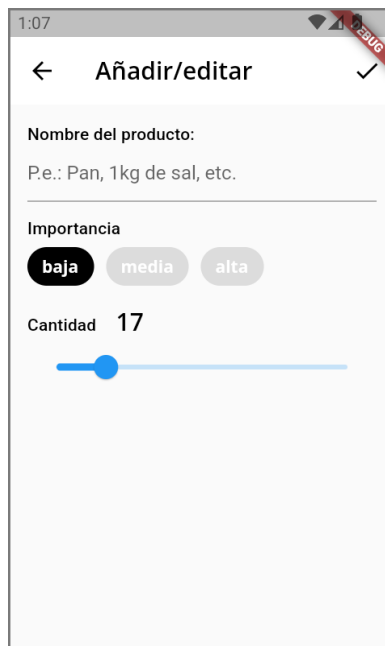
Por último hay que mostrar todas estas vistas en la pantalla `ListaCompraAnadirProducto` . Para ello, en el método `build` de la clase `_ListaCompraAnadirProductoState` , justo a continuación de:

```
construyeCampoImportancia(),
```

hay que añadir:

```
const SizedBox(height: 8,),  
construyeCampoCantidad(),
```

Con todo eso, la pantalla completa para añadir/editar productos a la lista de la compra queda así:



Pantalla para añadir/editar productos de la lista de la compra completa.

16 – Añadir los productos a la lista de la compra

La pantalla `ListaCompraAnadirProducto` donde se añaden/editan los productos de la lista de la compra es completamente funcional, pero todavía no desencadena que se añadan productos a la lista. Tal y como está en este punto del desarrollo, al pulsar el botón  simplemente se vuelve a la pantalla `ListaCompraPantalla` sin añadir nada.

Vamos a conectar el botón  con el `ChangeNotifier` al que hemos llamado `ListaCompra` para que cuando se pulse el botón se añada un nuevo producto a la lista.

Comenzaremos añadiendo una nueva dependencia al proyecto. Como ya se explicó, la clase `ListaCompra` tiene una propiedad llamada `id` a la que se asigna un identificador único. Para ello usaremos el paquete `Uuid` de Dart, por lo que hay que añadirlo como dependencia al proyecto.

Como sabemos, las dependencias de un proyecto Flutter se gestionan mediante el archivo `pubspec.yaml` . Añadiremos la línea:

```
uuid: ^3.0.7
```

justo a continuación de la línea:

```
provider: ^6.0.5
```

manteniendo el mismo sangrado.

A continuación salvaremos el archivo `pubspec.yaml` y ejecutaremos `flutter pub get` para actualizar las dependencias.

Una vez hecho esto, modificaremos la función anónima asociada a la propiedad `onPressed:` del botón de la pantalla `ListaCompraAnadirProducto` (el que está marcado con un tick mark ) , que por ahora solo hace que se vuelva a la pantalla anterior. Debe construir un nuevo objeto de tipo `Producto` con los datos que ha proporcionado el usuario y pasarlo a la `ListaCompra` que es quien se encarga de gestionar la lista de la compra, para que esta lo añada.

Para ello buscaremos dentro del método `build` de la clase `_ListaCompraAnadirProductoState` la propiedad `onPressed:` del `IconButton` que en este momento tiene asociada la función anónima:

```
onPressed: () {  
  Navigator.pop(context);  
},
```

y la cambiaremos por lo siguiente:

```
onPressed: () {  
  final producto = Producto(  
    id: widget.productoOriginal?.id ?? const Uuid().v1(),  
    nombre: _controladorNombre.text,  
    importancia: _importancia,  
    cantidad: _valorActualSlider,  
  );  
  
  if (widget.actualizando) {  
    widget.editarProducto(producto);  
  } else {  
    widget.crearProducto(producto);  
  }  
},
```

Veamos algunas consideraciones sobre este código:

- En primer lugar se construye un nuevo objeto de la clase `Producto` que contendrá los valores dados por el usuario y que ahora están en las propiedades privadas de `_ListaCompraAnadirProductoState`
- En el caso de la propiedad `id` de la clase `Producto` hay que distinguir dos situaciones: se está añadiendo un nuevo elemento o se está modificando uno ya creado. Para ello se recurre a la propiedad `productoOriginal` de la clase `ListaCompraAnadirProducto` . Si es `null` es porque se está construyendo un nuevo producto.

En ese caso, se llama a:

```
Uuid().v1()
```

que genera una cadena de caracteres que es un identificador único de tipo `v1`, es decir, basado en la hora a la que se genera. El resultado obtenido es una cadena similar a esta:

```
'6c84fb90-12c4-11e1-840d-7b25c5ee775a'
```

Nota: El paquete UUID tiene otros modos para generar identificadores únicos. Para una mayor información se puede consultar la documentación en el siguiente enlace:

[Documentación paquete UUID](#)

Nota: La propiedad `productoOriginal` pertenece a la clase `ListaCompraAnadirProducto` pero se usa en la clase que contiene su estado asociado `_ListaCompraAnadirProductoState`, por eso se accede a través de la propiedad `widget` de la clase `State`

- Una vez construido el objeto `Producto` con los valores correctos de las propiedades se llama al callback apropiado `editarProducto` o `crearProducto` según sea el valor de la propiedad `actualizando` de la clase `ListaCompraAnadirProducto`

El callback `crearProducto` que se pasa en la pantalla `ListaCompraPantalla` por ahora solo hace que se vuelva a la pantalla anterior. Para que los productos se añadan realmente a la lista de la compra hay que modificarlo.

En la clase `ListaCompraPantalla` buscaremos las líneas:

```
onPressed: () {  
  Navigator.push(  
    context, MaterialPageRoute(builder: (context) => ListaCompraPantalla()),  
  );  
}
```

y las sustituiremos por:

```
onPressed: () {  
  final manager = Provider.of<ListaCompra>(context, listen: false);  
  Navigator.push(  
    context, MaterialPageRoute(builder: (context) => ListaCompraPantalla()),  
  );  
  manager.addItem(producto);  
}
```

Y después buscamos la línea:

```
crearProducto: (producto) { Navigator.pop(context); },
```

y la sustituimos por:

```
crearProducto: (producto) {  
  manager.addItem(producto);  
  Navigator.pop(context);  
},
```

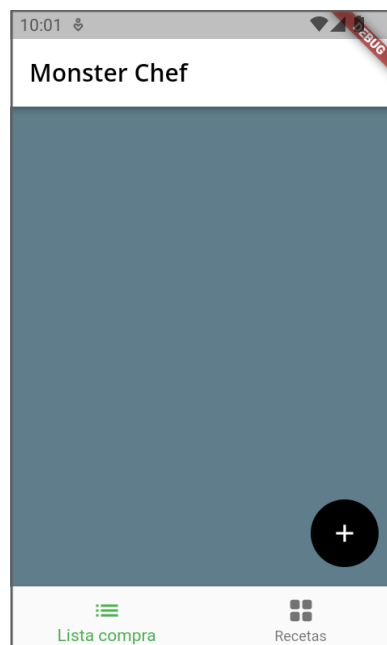

Veamos algunas consideraciones sobre el código anterior:

- En este caso, se accede al `ChangeNotifierProvider` que encapsula al `ChangeNotifier` al que hemos llamado `ListaCompra` solo para acceder a sus métodos y ejecutar las acciones para añadir un nuevo producto a la lista, no para observar sus cambios. Por eso no se usa un `Consumer` que es un observador de los cambios, sino `Provider.of<ListaCompra>`

Nota: También es por eso que se pasa `false` a la propiedad `listen:`

- El callback que se pasa a la propiedad `crearProducto:` será ejecutado cuando el usuario pulse el botón  de la pantalla `ListaCompraAnadirProducto` por tanto deberá hacer dos cosas:
 - Añadir un nuevo producto a la lista de la compra.
 - Hacer que se vuelva a la pantalla `ListaCompraPantalla`

Si hacemos hot reload, añadimos un nuevo producto y tocamos en el botón etiquetado como  veremos que el resultado es:



Resultado tras añadir un nuevo producto a la lista de la compra. No se vuelve a la pantalla vacía `ListaCompraPantallaVacía` sino que se muestra la pantalla para cuando la lista de la compra tiene productos que todavía no está completa.

Como puede verse, ya no se vuelve a la pantalla `ListaCompraPantallaVacía` porque la lista ya no está vacía (se acaba de añadir un producto). Se muestra una pantalla con el fondo gris azulado porque aún no se ha construido la pantalla `ListaCompraPantallaLlena` y en su lugar, en el método `construirPantallaListaCompra` de la clase `ListaCompraPantalla` se usa una clase `Container` con color de fondo `Colors.blueGrey`

```
if (manager.productos.isNotEmpty) {  
  return Container( color: Colors.blueGrey, );  
} else {  
  return const ListaCompraPantallaVacía();  
}
```

Cambiaremos eso a continuación.

17 – Pantalla para mostrar la lista de la compra con productos

El siguiente paso es añadir una pantalla para mostrar los productos que haya en la lista de la compra. Esta pantalla se mostrará en lugar de la pantalla `ListaCompraPantallaVacía` cuando la lista de la compra no esté vacía.

Añadiremos un nuevo archivo al proyecto llamado `lista_compra_pantalla_llena.dart` con el siguiente código:

```
class ListaCompraPantallaLlena extends StatelessWidget {
  const ListaCompraPantallaLlena({Key? key, required this.listaCompra}) : super(key:
  key);

  final ListaCompra listaCompra;

  @override
  Widget build(BuildContext context) {
    final productos = listaCompra.productos;
    return Padding(
      padding: const EdgeInsets.all(10.0),
      child: ListView.separated(
        itemCount: productos.length,
        separatorBuilder: (context, index) {
          return const SizedBox(height: 8.0);
        },
        itemBuilder: (context, index) {
          final producto = productos[index];
          return Text(
            producto.nombre,
          );
        },
      ),
    );
  }
}
```

Veamos algunos aspectos interesantes de esta clase:

- Para representar la lista de la compra se usa un objeto de la clase `ListView` como el que se han usado en la pantalla para añadir nuevos productos. La diferencia es que en este caso usaremos el constructor con nombre llamado `separated` que construye un array de longitud fija de vistas, separadas entre sí por otras vistas que actúan como separadores, sobre el que se puede hacer scroll.

Nota: Este constructor es apropiado para listas de vistas con un gran número de elementos y separadores porque los constructores solo se llaman para los hijos que son realmente visibles.

Se usan tres propiedades del constructor `separated` la clase `ListView`

- `itemCount`: que indica el número de elementos que se muestran en el `ListView`. En este caso se usa el número de productos de la lista de la compra `productos.length`

- `itemBuilder`: recibe una función anónima que se ejecuta tantas veces como productos haya en la `ListaCompra`. Esa función anónima construye la vista que va a representar en la pantalla los datos de cada producto de la lista.

La función anónima que se pasa en `itemBuilder` tiene dos parámetros: `context` e `index` que indica el número del elemento de la lista de la compra para el que se está construyendo su vista en un momento dado.

Por ahora, para cada producto de la lista de la compra se construye una vista de clase `Text` en la que se muestra su nombre `producto.nombre`

Nota: Lo cambiaremos a continuación para que muestre más información relevante de los productos.

- `separatorBuilder` es un builder que indica cómo se construye la separación entre dos ítems consecutivos del `ListView`. En este caso, simplemente se construye un `SizeBox` con altura `8`

Para usar esta nueva pantalla hay que abrir el archivo `lista_compra_pantalla.dart`, buscar las líneas:

```
if (manager.productos.isNotEmpty) {  
  return Container( color: Colors.blueGrey, );  
} else {  
  return const ListaCompraPantallaVacía();  
}
```

y sustituirlas por las líneas:

```
if (manager.productos.isNotEmpty) {  
  return ListaCompraPantallaLlena(listaCompra: manager);  
} else {  
  return const ListaCompraPantallaVacía();  
}
```

Si se hace hot reload y se añaden algunos productos a la lista de la compra, ahora el resultado es similar a este:

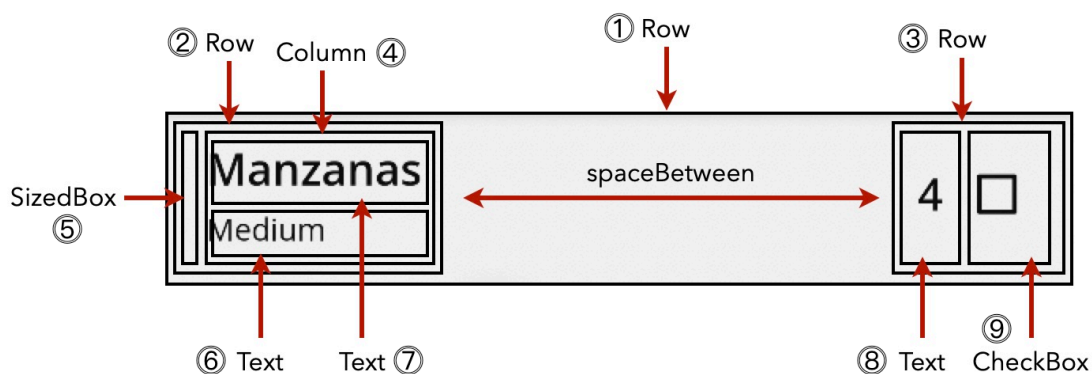


Aspecto de la aplicación tras añadir y conectar la clase `ListaCompraPantallaLlena`. Ahora se ve una lista de productos, aunque por ahora solo se muestra su nombre.

La pantalla con fondo gris azulado ha desaparecido y en su lugar ahora se muestra una lista con los productos añadidos. Por ahora solo se ve un texto con el nombre de cada producto y no responde a las acciones de pulsar y desplazar. En los próximos pasos se terminará la aplicación.

18 – Detallar la información de los productos

Para mostrar adecuadamente cada producto de la lista de la compra, usaremos la siguiente estructura de vistas:



Estructura de las vistas que representan a un producto en la lista de la compra.

Tiene muchas vistas dentro de vistas para permitir una correcta alineación y espaciado, aprovechando al máximo el tamaño de la pantalla del dispositivo. Es una estructura compleja de escribir, por eso, en lugar de pasar todas las vistas directamente a la función anónima de la propiedad `itemBuilder:` crearemos una clase personalizada para encapsular toda esa estructura y simplificar el constructor del `ListView`.

Comenzaremos añadiendo una nueva propiedad de tipo `Color` al tipo enumerado `Importancia`, llamaremos a esa propiedad `_color` y le añadiremos también un getter llamado `color`. Asignaremos el color `Colors.black` a las constantes del tipo enumerado `baja` y `media` y el color `Colors.red` a la constante `alta`

Nota: El proceso es idéntico al que se ha seguido anteriormente para añadir la propiedad `_nombre`

Una vez refactorizado el tipo enumerado, añadiendo un nuevo archivo al proyecto llamado `linea_producto.dart` y en él escribiremos la clase `LineaProducto` que representa a las vistas para mostrar en pantalla un producto concreto de la lista.

```
class LineaProducto extends StatelessWidget {
  final Producto producto;
  final TextDecoration textDecoration;

  LineaProducto({Key? key, required this.producto}) :
    textDecoration = producto.completado ? TextDecoration.lineThrough :
    TextDecoration.none,
    super(key: key);

  @override
  Widget build(BuildContext context) {
    var titleLarge = Theme.of(context).textTheme.titleLarge;
    var decortitleLarge = titleLarge?.copyWith(decoration: textDecoration);

    return Container(
      height: 60.0,
      child: Row( // - Número ① en el diagrama
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: <Widget>[
          Row( // - Número ② en el diagrama
            children: <Widget>[
              const SizedBox(width: 8,), // - Número ⑤ en el diagrama
              Column( // - Número ④ en el diagrama
                mainAxisAlignment: MainAxisAlignment.center,
                crossAxisAlignment: CrossAxisAlignment.start,
                children: <Widget>[
                  Text( // - Número ⑦ en el diagrama
                    producto.nombre,
                    style: decortitleLarge!,
                  ),
                  const SizedBox(height: 4.0),
                  construirImportancia(context, producto), // - Número ⑥ en el diagrama
                ],
              ),
            ],
          ),
        ],
      ),
    ),
  ),
),
```

(Continúa en la página siguiente)

```

Row( // - Número ③ en el diagrama
  children: <Widget>[
    Text( // - Número ⑧ en el diagrama
      producto.cantidad.toString(),
      style: titleLarge,
    ),
    Checkbox( // - Número ⑨ en el diagrama
      value: producto.completado,
      onChanged: (cambiado) {},
    ),
  ],
),
],
),
],
);
}

Widget construirImportancia(BuildContext context, Producto producto) {
  var temaTexto = Theme.of(context).textTheme.bodyMedium;
  var temaTextoColoreado = temaTexto?.copyWith(color: producto.importancia.color);
  return Text(
    producto.importancia.nombre,
    style: temaTextoColoreado!,
  );
}
}

```

Veamos algunas consideraciones sobre esta vista:

- El código anterior se ejecuta una vez por cada producto que haya en la lista de la compra. Se vuelve a ejecutar cada vez que hay algún cambio en la lista de la compra (se añade o borra un producto) o bien se edita alguno de los que ya hay.
- En la clase `LineaProducto` se ha añadido una propiedad llamada `textDecoration` que se inicializa con el valor `TextDecoration.lineThrough` (tachado) si el producto se ha marcado como completado (su propiedad `completado` es `true`) y con el valor `TextDecoration.none` si no está marcado como completado.

De esta forma, los productos de la lista de la compra ya comprados se podrán distinguir fácilmente de los pendientes, ya que además de tener el tick mark en el checkbox, su nombre estará tachado.

La propiedad `textDecoration` se inicializa en la lista de inicialización del constructor de `LineaProducto`.

```

textDecoration = producto.completado ? TextDecoration.lineThrough :
TextDecoration.none

```

- En la primera vista de clase `Row` (marcada con el número ① en el diagrama) que es la más externa y contiene a todas las demás, se ha usado el modo de alineación `MainAxisAlignment.spaceBetween`. Eso hace que el espacio que sobre después de construir las vistas hijas (en este caso otras dos vistas `Row`) se sitúe entre ellas, de forma que una quede completamente pegada a la izquierda y la otra a la derecha.

De esta forma siempre se va a ver bien independientemente del tamaño y resolución de la pantalla del teléfono en el que se ejecute.

- En la vista `Row` situada mas a la izquierda (Marcada con ② en el diagrama), la que contiene el texto para el nombre del producto y su importancia, se ha añadido un `SnackBar` con una anchura de 8 dp para que los textos no queden demasiado pegados al borde izquierdo de la pantalla.
- Las vistas que representan al campo `importancia` son un poco más complejas de construir, por eso se han encapsulado en un método de utilidad `construirImportancia` dentro de la propia clase `LineaProducto`

A este método se le pasa el contexto, para que se pueda acceder al tema común definido para la aplicación y el producto para el que se está construyendo su vista, para que pueda acceder al valor de la propiedad `importancia` y a su color asociado.

- Se quiere que si un producto tiene valor `Importancia.alta` para la propiedad `importancia` se muestre en color rojo. Eso se realiza automáticamente, ya que en el tipo enumerado que codifica los distintos valores para la importancia se ha añadido una propiedad `_color` con su correspondiente getter `color`

Para aplicarlo al texto se construye una variable `temaTextoColoreado` basada en el tema `bodyMedium` del contexto, al que se le ha aplicado el color correspondiente al valor de importancia que tenga ese producto:

```
var temaTexto = Theme.of(context).textTheme.bodyMedium;
var temaTextoColoreado = temaTexto?.copyWith(color: producto.importancia.color);
```

- El objeto de la clase `CheckBox` que muestra el valor de la propiedad `completado` de los productos, recibe una función anónima en su propiedad `onChanged`: Esta función anónima es la que se ejecuta cuando se pulsa en el `CheckBox` para cambiar su estado. Por ahora está vacía. Se completará en la próxima sección.
- Por ahora, la vista no responde a ninguna acción del usuario. En próximas secciones se añadirá el código necesario para editar, marcar como completada y borrar.

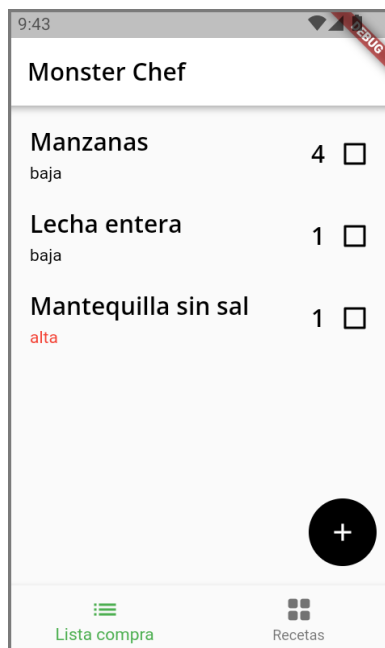
Para usar la vista `LineaProducto` recién programada, hay que buscar en la clase `ListaCompraPantallaLlena` las líneas:

```
return Text(
  producto.nombre,
);
```

y sustituirlas por la línea:

```
return LineaProducto( producto: producto, );
```

Si se hace hot reload y se añaden algunos productos a la lista de la compra, ahora el resultado que se muestra es:



Aspecto de la aplicación tras terminar la clase `ListaCompraPantallaLlena`. Ahora se ve una lista de productos con toda la información asociada a cada uno.

19 – Marcar una tarea como completada

En este paso vamos a hacer que el `CheckBox` que muestra la propiedad `completado` de un `Producto` (producto de la lista de la compra) sea funcional, es decir, que cuando se pulse, su estado cambie. Si estaba no marcado que se marque y viceversa.

Para ello añadiremos una nueva propiedad llamada `completar` a la clase `LineaProducto`

```
final Function(bool?)? completar;
```

y en el constructor de `LineaProducto` añadiremos:

```
required this.completar,
```

A continuación en el `CheckBox` buscaremos la línea:

```
onChanged: (change) {},
```

y la sustituiremos por la línea:

```
onChanged: completar,
```

Ahora abriremos el archivo `lista_compra_pantalla_llena.dart` y buscaremos las líneas:

```
return LineaProducto( producto: producto, );
```


para sustituirlas por las líneas:

```
return LineaProducto(  
    producto: producto,  
    completar: (valor) {  
        if (valor != null) {  
            listaCompra.marcaCompletado(index, valor);  
        }  
    },  
);
```

Algunas consideraciones sobre el código anterior:

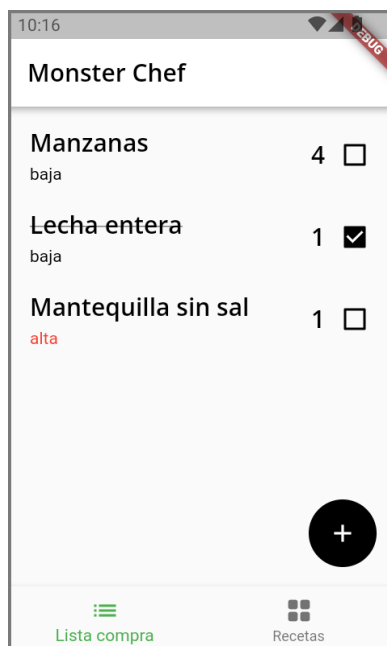
- La propiedad `onChanged:` de la vista `CheckBox` espera una función anónima de tipo `void Function(bool?)` por tanto la variable `completar` debe definirse de ese tipo. Esta función anónima se llama cuando el valor del `CheckBox` deba ser cambiado. La propia clase `CheckBox` pasa el nuevo valor a la función anónima, pero en realidad no cambia su estado interno hasta que no se reconstruya el `CheckBox` con el nuevo valor.

Por eso, el contenido de esa función invoca al método `marcaCompletado` de `ListaCompra` para que actualice la lista de productos y notifique que se debe reconstruir el árbol de vistas completo. Eso se consigue con la orden:

```
listaCompra.marcaCompletado(index, valor);
```

- `valor` es de tipo `bool?` por tanto podría ser `null`, para que compile y ejecute es necesario el `if` que comprueba que no se tenga el valor `null`

Si hacemos hot reload y en la lista de la compra, tocamos el `CheckBox` de alguno de los productos, vemos como su estado cambia.



Efecto de tocar el check box de uno de los productos de la lista de la compra.

20 – Editar un producto ya añadido

Ahora vamos a añadir el comportamiento para que al pulsar sobre un producto en la lista de la compra, se muestre de nuevo la pantalla `ListaCompraAnadirProducto` esta vez para poder modificarlo. En esta ocasión usaremos la clase `InkWell` para detectar el gesto de pulsar y realizar la animación.

En primer lugar editaremos el archivo `lista_compra_anadir_producto.dart`. Buscaremos el método `initState` de la vista `_ListaCompraAnadirProductoState` y justo debajo de `super.initState();` añadiremos lo siguiente:

```
final productoOriginal = widget.productoOriginal;
if (productoOriginal != null) {
  _controladorNombre.text = productoOriginal.nombre;
  _nombre = productoOriginal.nombre;
  _valorActualSlider = productoOriginal.cantidad;
  _importancia = productoOriginal.importancia;
}
```

Vamos a estudiarlo con detalle:

- Si el producto original es `null` es porque se está añadiendo un producto nuevo, mientras que si no lo es, es porque se está editando uno ya creado. De esta forma se pueden distinguir los dos casos y se puede utilizar la misma pantalla para añadir nuevos productos y editar productos ya añadidos.
- En caso de estar en modo edición se preparan las propiedades de `_ListaCompraAnadirProductoState` de forma que al construir las vistas que muestran los datos del producto se usen los valores de las propiedades del producto que esté seleccionado y no los valores por defecto.

A continuación abriremos el archivo `lista_compra_pantalla_llena.dart` y en él buscaremos dentro del método `build` las líneas:

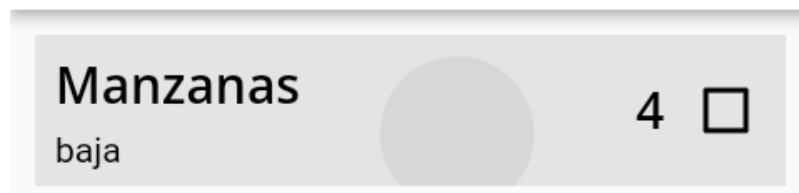
```
return LineaProducto(
  producto: producto,
  completar: (valor) {
    if (valor != null) {
      listaCompra.marcaCompletado(index, valor);
    }
  },
);
```

y las sustituiremos por las líneas:

```
return InkWell(  
  child: LineaProducto(  
    producto: producto,  
    completar: (valor) {  
      if (valor != null) {  
        listaCompra.marcaCompletado(index, valor);  
      }  
    },  
  ),  
  onTap: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(  
        builder: (context) => ListaCompraAnadirProducto(  
          productoOriginal: producto,  
          editarProducto: (producto) {  
            listaCompra.actualizaProducto(producto, index);  
            Navigator.pop(context);  
          },  
          crearProducto: (producto) {},  
        ),  
      ),  
    ),  
  );  
};
```

Veamos el código anterior con detalle:

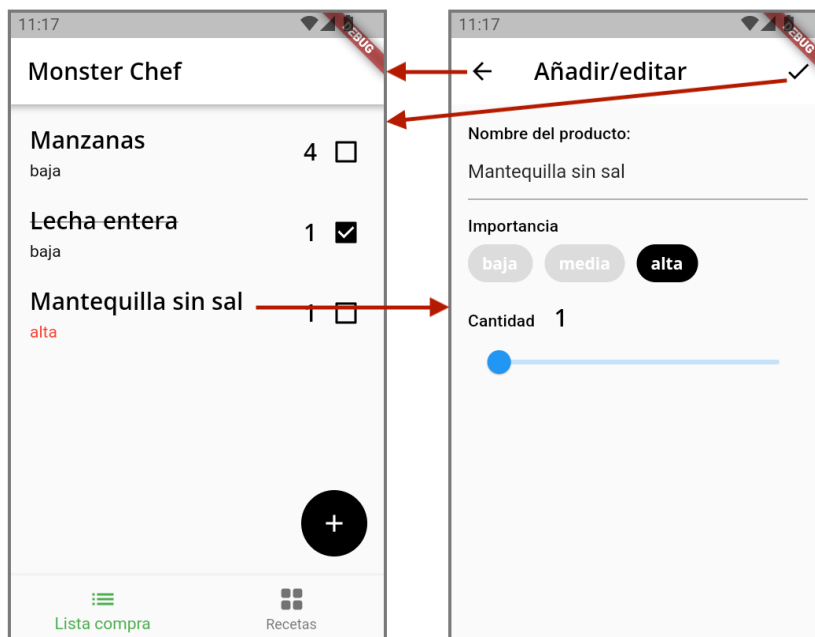
- Como puede apreciarse se ha envuelto la vista `LineaProducto` dentro de una vista `InkWell`. Esta vista responde a los toques del usuario y realimenta la acción mediante una animación que simula el efecto de una gota de tinta que cae sobre la vista.



Efecto de gota de tinta proporcionado por una vista `InkWell`

- De las muchas propiedades que incluye la vista `InkWell` en este caso se han usado dos:
 - `child`: en la que se ha incluido la `LineaProducto`. Es lo que se muestra dentro del `InkWell`.
 - `onTap`: que acepta una función anónima sin parámetros, en la que se indica cómo debe comportarse la vista cuando se pulsa sobre ella. El comportamiento es muy similar al ya estudiado para crear nuevos objetos. La diferencia es que en este caso el callback que se pasa vacío es el de la propiedad `crearProducto` ya que no se están creando nuevos productos, sino editando uno ya añadido previamente a la lista de la compra.

Si hacemos hot reload, y tocamos uno de los productos ya añadidos de la lista de la compra, vemos como se muestra la pantalla para añadirlo/editarlo, con sus valores de propiedad ya cargados. Si se pulsa el botón para cancelar (la flecha) los datos no se cambian y si se pulsa el botón para aceptar (el tick mark) los cambios se guardan en la lista de la compra.



Al tocar en un producto de la lista de la compra se muestra la pantalla para crear/editar productos y se pueden modificar sus propiedades. Si se pulsa en la flecha se cancela la edición y si se pulsa en el tick mark se acepta.

21 – Eliminar un producto

La última acción que se va a implementar es la de eliminar un producto de la lista deslizando hacia la izquierda sobre ese producto. Para ello usaremos una vista de tipo `Dismissible` y dentro de ella envolveremos a la vista `InkWell` construída en el paso anterior.

Para ello en el método `build` de la clase `ListaCompraPantallaLlena` buscaremos las líneas:

```
return InkWell(  
  child: LineaProducto(  
    // - El resto del contenido se ha ocultado  
  ),  
  // - El resto del contenido se ha ocultado  
  onTap: () {  
    Navigator.push(  
      context,  
      // - El resto del contenido se ha ocultado  
    );  
  });  
);
```

y las sustituiremos por las líneas:

```
return Dismissible(  
  key: Key(producto.id),  
  direction: DismissDirection.endToStart,  
  background: Container(  
    color: Colors.red,  
    alignment: Alignment.centerRight,  
    child: const Icon(  
      Icons.delete_forever,  
      color: Colors.white,  
      size: 35.0  
    ),  
  ),  
  onDismissed: (direction) {  
    listaCompra.borraProducto(index);  
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(  
        content: Text('${producto.nombre} borrado'),  
      ),  
    ),  
  },  
  child: InkWell(  
    key: Key(producto.id),  
    child: LineaProducto(  
      producto: producto,  
      completar: (valor) {  
        if (valor != null) {  
          listaCompra.marcaCompletado(index, valor);  
        }  
      },  
    ),  
    onTap: () {  
      Navigator.push(  
        context,  
        MaterialPageRoute(  
          builder: (context) => ListaCompraAnadirProducto(  
            productoOriginal: producto,  
            editarProducto: (producto) {  
              listaCompra.actualizaProducto(producto, index);  
              Navigator.pop(context);  
            },  
            crearProducto: (producto) {},  
          ),  
        ),  
      ),  
    ),  
  ),  
);
```

Algunas consideraciones sobre el código anterior:

- Hemos envuelto la vista `InkWell` dentro de una vista `Dismissible`. Esta vista acepta acciones de deslizar y permite descartar la vista con una animación. Es muy apropiado para implementar operaciones de borrado, porque la animación permite al usuario percibir claramente que ese elemento del `ListView` se ha eliminado de la lista.

- Un cambio que llama la atención es que se usa el parámetro `key:` tanto en la vista `Dismissible` como en la vista `InkWell` :

```
key: Key(producto.id),
```

Como valor para `key:` se usa el `id` del producto, que como sabemos se inicializa a una constante única para cada producto, y es de tipo `String`. Para eso se usó el paquete `Uuid`.

En las vistas de tipo `Dismissible` el parámetro `key:` es obligatorio, no puede ser nulo. La razón es que las vistas `Dismissible` se usan por lo general en listas y se eliminan de la lista cuando se descartan.

El comportamiento predeterminado de `ListView` es sincronizar las vistas que representan a cada fila en función de su índice en la lista. Si no se eliminan filas no hay problema, pero si se elimina una fila, la fila posterior a la fila descartada se sincronizaría con el estado del producto que ocupaba la fila descartada, lo que es incorrecto. El uso del parámetro `key:` hace que los widgets se sincronicen de acuerdo con sus valores de la propiedad `key:` y no por su índice, por lo que no hay ningún problema.

- La propiedad `direction:` de la clase `Dismissible` es de tipo `DismissDirection`. Permite limitar el tipo de los deslizamientos que se permiten sobre la vista. Si no se indica nada, se puede deslizar hacia arriba, hacia abajo, hacia la derecha y hacia la izquierda.

En esta aplicación solo se permiten deslizamientos hacia la izquierda, por eso, se le ha dado el valor `DismissDirection.endToStart`.

Nota: Si se quieren deslizamientos hacia la izquierda y hacia la derecha, pero no hacia arriba y hacia abajo, se puede dar el valor `horizontal` a esta propiedad.

- La propiedad `background` de la vista `Dismissible` permite cambiar el color del fondo, colocar un icono, o cualquier otra vista que se quiera para que al deslizar se indique visualmente la acción que se está llevando a cabo. En este caso se usa un `Container` con fondo de color rojo y un icono blanco de una papelera.
- En la propiedad `onDismissed:` se pasa la función anónima que describe el comportamiento que se va a tener cuando se realice un deslizamiento. La función tiene un parámetro llamado `direction` que es rellenado por la propia vista `Dismissible`. Si se han permitido varios deslizamientos en la vista, se debe usar este parámetro para comprobar cual se ha producido y realizar la acción correspondiente.

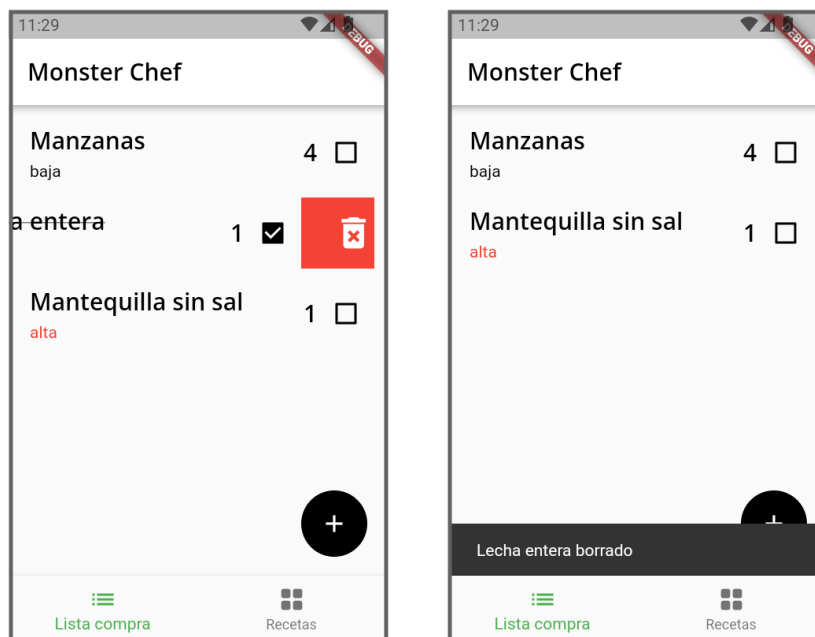
La función anónima llama al método `borraProducto` de la `ListaCompra` pasándole como argumento `index` que indica el índice del elemento de la lista de productos que hay que eliminar.

- Se ha añadido un snack en la `SnackBar` (la barra temporal que se muestra justo por encima de la `BottomNavigationBar`) para informar al usuario de la acción de borrado. Para acceder a la `SnackBar` se usa `ScaffoldMessenger.of(context)` que permite acceder desde las vistas inferiores del árbol de vistas a la vista `Scaffold` en la que están contenidas.

La `SnackBar` desaparece automáticamente al cabo de un tiempo predeterminado que puede modificarse con su propiedad `duration:`

En este caso la `SnackBar` solo tiene una vista `Text` indicando que se acaba de eliminar un elemento. Se pueden añadir también otras vistas, como por ejemplo un botón para deshacer el borrado si se ha producido por error.

Si se hace hot reload y se desliza hacia la izquierda sobre alguno de los productos de la lista de la compra, el producto se borra.



Al deslizar hacia la izquierda se elimina un elemento de la lista. Justo después se muestra un `SnackBar` en la parte inferior indicando la acción realizada.

Con esto se acaba la programación de la aplicación para gestionar una lista de la compra. Aún le quedan muchas mejoras que se pueden hacer. La principal es guardar la lista de la compra. Tal y como está, la lista de la compra se almacena en memoria RAM. Cuando se sale de la aplicación, su contenido se pierde. Hay que guardarla en disco para poder seguir utilizándola posteriormente. Para saber cómo hacer esto, mira los ejercicios.

Mini ejercicios

1. ➡ La lista de la compra tal y como está no proporciona la mejor experiencia de uso. Hay pasos que se podrían evitar al usuario. Identifica esos pasos superfluos (Exise puro) e indica qué se podría hacer para resolverlos.
2. ➡ En la lista de la compra tal y como está, también hay algunas acciones de interacción que son difíciles de realizar, lo que hace que la experiencia de uso sea menor. Identifica esas acciones e indica qué se podría hacer para mejorar la interacción.
3. ➡ Añade a la aplicación la capacidad de observar un nuevo gesto. Cada vez que se deslice hacia la derecha sobre un producto de la lista de la compra, se cambiará su propiedad `_importancia` al valor `Importancia.alta`
4. ➡ ¿Qué se está definiendo en esta línea de código?

```
final Function(bool?)? completar;
```

5. ➡ Añade a la aplicación la capacidad de guardar la lista de la compra para futuras ejecuciones.

Pista: Mira la documentación de Flutter sobre cómo leer y guardar datos en archivos:

<https://esflutter.dev/docs/cookbook/persistence/reading-writing-files>

Apéndice A: Listado completo del tema personalizado

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

class MonsterChefTema {
  static TextTheme temaTextoClaro = TextTheme(
    bodyLarge: GoogleFonts.openSans(
      fontSize: 14.0,
      fontWeight: FontWeight.w700,
      color: Colors.black,
    ),
    displayLarge: GoogleFonts.openSans(
      fontSize: 32.0,
      fontWeight: FontWeight.bold,
      color: Colors.black,
    ),
    displayMedium: GoogleFonts.openSans(
      fontSize: 21.0,
      fontWeight: FontWeight.w700,
      color: Colors.black,
    ),
    displaySmall: GoogleFonts.openSans(
      fontSize: 16.0,
      fontWeight: FontWeight.w600,
      color: Colors.black,
    ),
    titleLarge: GoogleFonts.openSans(
      fontSize: 20.0,
      fontWeight: FontWeight.w600,
      color: Colors.black,
    ),
  );

  static TextTheme temaTextoOscuro = TextTheme(
    bodyLarge: GoogleFonts.openSans(
      fontSize: 14.0,
      fontWeight: FontWeight.w700,
      color: Colors.white,
    ),
    displayLarge: GoogleFonts.openSans(
      fontSize: 32.0,
      fontWeight: FontWeight.bold,
      color: Colors.white,
    ),
    displayMedium: GoogleFonts.openSans(
      fontSize: 21.0,
      fontWeight: FontWeight.w700,
      color: Colors.white,
    ),
    displaySmall: GoogleFonts.openSans(
      fontSize: 16.0,
      fontWeight: FontWeight.w600,
      color: Colors.white,
    ),
    titleLarge: GoogleFonts.openSans(
      fontSize: 20.0,
      fontWeight: FontWeight.w600,
      color: Colors.white,
    ),
  );
};
```

```

static ThemeData claro() {
  return ThemeData(
    brightness: Brightness.light,
    checkboxTheme: CheckboxThemeData(
      fillColor: MaterialStateColor.resolveWith((states) {
        return Colors.black;
      }),
    ),
  ),
  appBarTheme: const AppBarTheme(
    foregroundColor: Colors.black,
    backgroundColor: Colors.white,
  ),
  floatingActionButtonTheme: const
  FloatingActionButtonThemeData(
    foregroundColor: Colors.white,
    backgroundColor: Colors.black,
  ),
  bottomNavigationBarTheme: const
  BottomNavigationBarThemeData(
    selectedItemColor: Colors.green,
  ),
  textTheme: temaTextoClaro,
);
}

static ThemeData oscuro() {
  return ThemeData(
    brightness: Brightness.dark,
    checkboxTheme: CheckboxThemeData(
      fillColor: MaterialStateColor.resolveWith((states) {
        return Colors.white;
      }),
    ),
  ),
  appBarTheme: const AppBarTheme(
    foregroundColor: Colors.white,
    backgroundColor: Colors.black26,
  ),
  floatingActionButtonTheme: const
  FloatingActionButtonThemeData(
    foregroundColor: Colors.white,
    backgroundColor: Colors.green,
  ),
  bottomNavigationBarTheme: const
  BottomNavigationBarThemeData(
    selectedItemColor: Colors.green,
  ),
  textTheme: temaTextoOscuro,
);
}
}

```